

Smart Teacher — Architecture & Scores Reference (Detailed Edition)

Reference complet : architecture, **calculs de scores avec formules + exemples chiffrés**, table de tous les paramètres **Config**, API REST + WebSocket avec corps de messages, légende des logs avec extraits de sessions, deployment, troubleshooting. Version : 2026-05-04. Couvre Q&A graph, Teaching graph, RAG multimodal, Knowledge Graph, personnalisation (bandit Thompson + profil + confusion fusion), pause/resume intelligent, intégration Groq, anti-écho audio, observabilité.

Sommaire

Partie I — Vue d'ensemble 1. Glossaire 2. Vue d'ensemble 3. Services & dépendances externes 4. Authentification 5. Machine d'état dialogue (FSM)

Partie II — Pipelines 6. Q&A Graph 7. Teaching Graph 8. RAG multimodal 9. Knowledge Graph

Partie III — Pédagogie 10. Personnalisation 11. Pause / Resume intelligent 12. Détection de confusion 13. Adaptation du débit de parole

Partie IV — Calculs de score 14. Mastery (Beta-Binomial + Laplace) 15. FSRS — Spaced Repetition 16. Bandit Thompson sampling 17. Bandit reward 18. Confusion fusion 19. Engagement 20. RAG ranking (cosine, BM25, RRF, rerank) 21. Concept aggregation (snapshot) 22. Cursor / sentence-span

Partie V — Référence 23. Configuration 24. API REST 25. WebSocket protocol 26. Logs (référence complète)

Partie VI — Opérations 27. Deployment 28. Performance & SLA 29. Troubleshooting / FAQ

PARTIE I — VUE D'ENSEMBLE

1. Glossaire

Terme	Définition
RI	Recherche d'information (= Information Retrieval)
STT	Speech-to-Text (Whisper)
TTS	Text-to-Speech (Edge-TTS, ElevenLabs)
VAD	Voice Activity Detection (Silero)
RAG	Retrieval-Augmented Generation
BM25	Best Matching 25 — fonction de scoring lexicale (Robertson 1995)
RRF	Reciprocal Rank Fusion (Cormack 2009)
BGE	BAAI General Embeddings (modèles HuggingFace)
FSRS	Free Spaced Repetition Scheduler

Terme	Définition
KG	Knowledge Graph
FSM	Finite State Machine
WS	WebSocket
JWT	JSON Web Token
Bandit	Multi-armed bandit (algorithme de décision sous incertitude)
Thompson sampling	Stratégie bayésienne pour bandit (sample posterior, pick argmax)
SIGHT	Confusion classifier (XLM-Roberta fine-tuné, Bhatti 2022)
VARK	Visual / Auditory / Read-Write / Kinesthetic learning styles (Fleming 2001)
Bloom	Taxonomie de Bloom (remember/understand/apply/analyze/evaluate/create)
Groq	Provider LLM hosted (inference Llama-3.x très rapide)
Edge-TTS	TTS gratuit Microsoft (couvre 70+ voix multilingues)
Idea	Unité atomique de contenu pédagogique (déf, théorème, exemple, ...)
Concept	Cluster d'ideas portant le même <code>section_title</code>
Mastery	Probabilité postérieure que l'étudiant a maîtrisé une idée
Cursor	Position char dans une narration (0..len(text))
Snapshot	État cached d'une slide (text + cursor) en Redis

2. Vue d'ensemble

Smart Teacher est un **tuteur IA vocal** qui : 1. Présente un cours PDF section par section (narration TTS) 2. Détecte la confusion vocale + textuelle (SIGHT + prosodie) 3. Répond aux questions de l'étudiant (Q&A LangGraph + RAG) 4. Adapte le rythme de parole, le style pédagogique, et le contenu 5. Mémoire la mastery par concept (Beta-Laplace) et planifie la révision (FSRS) 6. Tolère pauses + interruptions + navigation (resume intelligent)

2.1 Modules de premier niveau

Module	Fichiers principaux	Rôle
<code>agentic/qa/</code>	<code>intent.py</code> , <code>rewriter.py</code> , <code>retriever.py</code> , <code>responder.py</code> , <code>reviewer.py</code> , <code>graph.py</code>	Graphe Q&A LangGraph
<code>agentic/teaching/</code>	<code>planner.py</code> , <code>context.py</code> , <code>adaptation.py</code> , <code>narrator.py</code> , <code>reviewer.py</code> , <code>graph.py</code>	Graphe narration LangGraph
<code>agentic/state.py</code>	TutorState	Pydantic state partagé entre nodes

Module	Fichiers principaux	Rôle
agentic/resilience/	wrap.py, fallbacks.py, circuit_breaker.py	Retry + fallback + circuit breaker
ai/	llm_router.py, llm.py, local_llm.py, prompt_rules.py	Brain + LLM router (OpenAI/Groq/Ollama)
audio/	transcriber.py, tts.py, voice/	STT Whisper + TTS Edge/EL + prosody
core/	config.py, diagnostics.py, domains_config.py	Config centralisée
database/	models.py, init_db.py, crud.py	SQLAlchemy + lazy migrations
handlers/	ws.py (3 800 lignes), audio_pipeline.py, auth.py, session_manager.py	WebSocket FSM + auth
pedagogy/	mastery_repo.py, dialogue.py, review_ scheduler.py, engagement.py, knowledge_graph/ personalization/ confusion/, resume_i ntelligence.py, stude nt_knowledge.py	Couche pédagogique
rag/	multimodal_rag.py, e mbedding_cache.py, evaluation/	RAG + ingestion
services/	presentation.py, vision_describe.py, nav_dispatcher.py, course_slides.py, text_turns.py	Services métier
routes/	auth.py, course.py, session.py, student.py, dashboa rd_services.py, admin.py, ...	Endpoints FastAPI
observability/	logger.py, analytics.py, dashboard.py, kpi_logger.py	Logs + métriques
storage/	media_storage.py, transcript_search.py, media_helpers.py	MinIO + ES

2.2 Flux d'une session typique

[Browser]

[Smart Teacher]

```
■ GET /static/index.html
■
■   ■■ Auth gate middleware
■       ■■ Cookie smart_teacher_token présent ?
■       ■■ JWT decode OK ?
■       ■■ NO → redirect 302 /static/login.html
■       ■■ YES → GET 200 OK
■
■   ■■ POST /auth/login {email, password}
■       ■■ Rate-limit check (5/5min)
■       ■■ DB lookup student
■       ■■ bcrypt.checkpw (~180ms)
■       ■■ create_access_token (HS256, 24h)
■       ■■ Set-Cookie smart_teacher_token (HttpOnly, SameSite=Lax)
■
■   ■■ WebSocket /ws/{uuid}
■       ■■ JWT extracted from cookie or header → decode
■
■   ■■ {type:"start_session", course_id, language, level}
■       ■■ dialogue.create_session() → Redis SETEX session:{uuid}
■       ■■ Postgres INSERT learning_sessions
■       ■■ State : IDLE → LISTENING
■
■   ■■ {type:"present_section", chapter_index, section_index, slide_content, ...}
■       ■■ services/presentation.decide_narration_cache_reuse(...)
■           ■■ Redis GET presentation:snapshot:{uuid}:{slide_id}
■           ■■ memory hit? → reuse (0 LLM call)
■           ■■ Redis snapshot? → reuse (0 LLM call)
■           ■■ MISS → Brain.present (Groq llama-3.3-70b ~1-3s)
■       ■■ Sentence-by-sentence TTS Edge-TTS
■           ■■ Redis GET presentation:tts:{md5} → cache hit ?
■           ■■ MISS → Edge-TTS WebSocket synthesis (~1-3s/sentence)
■           ■■ Redis SETEX cache (TTS_CACHE_TTL=24h)
■       ■■ Stream {type:"narration_chunk", seq, text, audio_b64}
■       ■■ State : LISTENING → PRESENTING → LISTENING (after end)
■
■   ■■ {type:"interrupt", reason:"pause", audio_progress:0.42, turn_id}
■       ■■ compute_text_cursor_from_audio_progress(0.42, len=1918) = 805
■       ■■ Override current_presentation_cursor = 805
■       ■■ dialogue.pause_session()
■           ■■ ctx.paused_state = {is_paused: True, slide_id, cursor: 805, timestamp, ...}
■           ■■ Redis SETEX session (1h)
■           ■■ Redis SETEX presentation:snapshot (1h)
■       ■■ start_pause_progress_ticker()
■       ■■ State : PRESENTING → WAITING
■
■       ■■ WAITING tick @ 15s, 30s, 60s, 90s, 2m, 3m, 4m, ...
■
■   ■■ {type:"text_question", text:"qu'est-ce que la RI ?"}
■       ■■ Q&A graph LangGraph
■           ■■ IntentAgent (SIGHT → 0.85 → confusion_signal)
■           ■■ Rewriter (skip si déjà fait par intent)
■           ■■ Retriever (Qdrant + BM25 + RRF + rerank → 5 chunks)
■           ■■ Personalization layer
■               ■■ get_or_create_profile
■               ■■ Bandit Thompson → arm winner
■               ■■ Knowledge snapshot
■           ■■ Responder (LLM ~1-2s avec Groq)
■           ■■ Off-topic guardrail (overlap >= 0.08 ?)
■           ■■ Reviewer (LLM Self-RAG ~0.5s)
■           ■■ retry × N if not grounded, else END
■       ■■ Mastery update (Beta-Laplace) sur idées mentionnés
■       ■■ FSRS update sur concepts dependants
```

- ■■ Bandit reward $\rightarrow \alpha += r, \beta += (1-r)$
- ■■ TTS de la réponse + stream
- ■■ {type:"text_question", text:"reprendre"}
- ■■ Detected as resume trigger
- ■■ stop_pause_progress_ticker $\rightarrow$ log total_wait
- ■■ dialogue.resume_session()
- ■■ pause_duration = now - ctx.paused_state.timestamp
- ■■ State : WAITING $\rightarrow$ PRESENTING
- ■■ ResumeIntelligence.compose_resume_action
- ■■ pause < 10s $\rightarrow$ CONTINUE
- ■■ < 60s $\rightarrow$ REWIND_SENTENCE
- ■■ < 180s $\rightarrow$ REEXPLAIN_AND_CONTINUE
- ■■ $\geq 180s \rightarrow$ REEXPLAIN_AND_CONTINUE
- ■■ services/presentation.rewind_to_current_sentence_start
- ■■ TTS resume from new cursor (cache hit fréquent)

2.3 État côté backend

Stockage	Contenu	TTL
Postgres	students, courses, chapters, sections, learning_sessions, interactions, learning_events, student_profiles, student_mastery, student_mistakes, practice_question, practice_attempt, review_queue (FSRS), confusion_events, learning_gain_tests, vark_responses, ingested_asset, rag_chunks, concepts, bandit_posteriors	persistant
Redis	session:{uuid} (SessionContext blob JSON)	SESSION_TTL = 1h
	presentation:snapshot:{uuid}:{slide_id} (text + cursor)	1h
	presentation:tts:{md5(text+lang+rate+provider+voice)} (audio MP3 b64)	24h
	chat::{student_id}:{course_id} (history list)	30j

Stockage	Contenu	TTL
	<code>narration:{md5}</code> (narration cross-session)	7j
	<code>embedding_cache:{md5}</code> (BGE embeddings)	24h
	<code>qa_cache:{md5}</code> (Q&A response cache)	session
	<code>seen_ideas:{session_id}</code> (set)	session
	<code>login:fails:{email}</code>	5 min sliding window
	<code>session_token:{uid}</code> (one-time WS auth)	5 min
Qdrant	Collection <code>smart_teacher_multimodal__{embedding_hash}</code> (BGE-m3 dim 1024, COSINE)	persistant
Elasticsearch	Index <code>smart_teacher_transcripts</code> (full-text Q&A turns)	persistant (fallback RAM)
MinIO / <code>media/</code>	PDF uploads, slide PNG (200dpi), audio TTS turns (.mp3), transcripts (.json)	persistant
ClickHouse	<code>learning_events</code> (analytics OLAP)	persistant (fallback CSV+RAM)
Disque	<code>cache/vision_descriptions/</code> (LLM vision per slide), <code>cache/slide_titles/</code> (LLM titles), <code>data/multimodal_db/{docs,idea,summary}_cache.json</code>	persistant

3. Services & dépendances externes

Service	Port	Rôle	Required ?	Fallback
Postgres	5432	Données persistantes	■ critique	—
Redis	6379	Cache + sessions	■ critique	—
Qdrant	6333	Vecteurs RAG	recommandé	BM25 in-memory

Service	Port	Rôle	Required ?	Fallback
Ollama	11434	LLM local Mistral	optionnel	—
Groq API	HTTPS	LLM hosté llama-3.3-70b (~250 tok/s)	recommandé	Ollama
OpenAI API	HTTPS	LLM premium gpt-4o-mini	optionnel	Groq → Ollama
ElevenLabs	HTTPS	TTS premium	optionnel	Edge-TTS
Edge-TTS	WSS	TTS gratuit Microsoft	recommandé	—
Elasticsearch	9200	Recherche transcripts	optionnel	RAM
MinIO	9000	Stockage médias	optionnel	Disque local
ClickHouse	8123	Analytics OLAP	optionnel	CSV+RAM
Tesseract OCR	local bin	OCR images PDF	recommandé	—
LibreOffice	local bin	DOCX/PPTX → PDF	optionnel	—

3.1 Health checks au boot

SmartTeacher.Diagnostics log au démarrage :

```

■ Diagnostic démarrage des services de données :
  • Elasticsearch : ■ connecté (cluster=docker-cluster, status=yellow)
  • MinIO : ■ connecté (bucket=smart-teacher)
  • ClickHouse : ■ connecté (smart_teacher.learning_events)
  • Redis : ■ connecté (localhost:6379)
■ PG ping OK | took=12ms | url=postgresql+asyncpg://admin:***@localhost:5432/smart_teacher
■ QDRANT collection_check | name=... exists=True | dim=1024 distance=COSINE
■ Local embeddings ready (BAAI/bge-m3) — dim=1024
■ Reranker BAAI/bge-reranker-v2-m3 ready
LLMRouter : ■ Groq activé | model=llama-3.3-70b-versatile base=https://api.groq.com/openai/v1

```

4. Authentification

4.1 JWT structure

Header : {"alg":"HS256", "typ":"JWT"} Payload : {"sub":"<uuid>", "email":"...", "account_level":"student|teacher|admin", "iat":..., "exp":...} Signature : HMAC-SHA256(Config.JWT_SECRET_KEY)

TTL : JWT_EXPIRATION_HOURS = 24.

Transport : cookie smart_teacher_token (HttpOnly, SameSite=Lax, Max-Age=86400) OU header Authorization: Bearer

4.2 Hardening

- **bcrypt** (rounds=12) pour stockage des passwords (~180ms par check)
- **Password strength** au register : len≥8, alpha+digit, pas dans liste de 10 mots de passe communs
- **Rate limiting** : 5 échecs/5min par email (Redis sliding window key login:fails:{email})

- **Audit log CSV** : `logs/sec_audit.csv` (timestamp, event, actor, target, ip, ua, outcome, details)
- **Static auth gate** : middleware FastAPI qui exige le JWT pour `GET /static/*.html` (sauf `login.html`)

4.3 Sample request — register

POST /auth/register HTTP/1.1
Content-Type: application/json

```
{
  "email": "alice@example.com",
  "password": "Strong123",
  "first_name": "Alice",
  "last_name": "Doe",
  "preferred_language": "fr",
  "student_level": "lycée"
}
```

Response 201 :

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "student_id": "ce566ac0-...",
  "email": "alice@example.com",
  "account_level": "student",
  "first_name": "Alice"
}
```

+ Set-Cookie : `smart_teacher_token=eyJ...; HttpOnly; SameSite=Lax; Max-Age=86400`

4.4 Logs auth (extrait session login + page protégée)

```
■ LOGIN START | email=alice@example.com | ip=127.0.0.1 ua=Mozilla/5.0...
■ rate_limit CHECK | id=alice@example.com | failures=0/5 window=300s | allowed=True remaining=5
■ LOGIN DB LOOKUP | email=alice@example.com | student_found=True
■ verify_password | match=True | bcrypt compare took=187ms
■ rate_limit RESET | id=alice@example.com | counter cleared after success
■ JWT CREATE | sub=ce566ac0 email=alice@... role=student | algo=HS256 ttl=24h exp=2026-05-05T18:22:05 | token_p
■ LOGIN SET-COOKIE | name=smart_teacher_token httponly=True samesite=lax max_age=86400s
■ AUDIT | event=login actor=ce566ac0 target=alice@... ip=127.0.0.1 outcome=success | role=student
■ LOGIN OK | sub=ce566ac0 email=alice@... role=student
```

(plus tard, sur `GET /static/index.html`)

```
■ GATE check | path=/static/index.html | token_source=cookie present=True
■ JWT DECODE OK | sub=ce566ac0 email=alice@... role=student exp=2026-05-05T18:22:05
■ GATE PASS | path=/static/index.html | sub=ce566ac0 role=student
```

4.5 Flux d'échec (rate limit)

```
■ LOGIN START | email=attacker@x.com | ip=1.2.3.4
■ rate_limit CHECK | id=attacker@x.com | failures=5/5 | allowed=False remaining=0
■ LOGIN BLOCKED | email=attacker@x.com | reason=rate_limited
■ AUDIT | event=login_rate_limited target=attacker@x.com ip=1.2.3.4 outcome=failure
```

HTTP 429 retourné avec `Retry-After: 300`.

5. Machine d'état dialogue (FSM)

Module : `pedagogy/dialogue.py`. Implémentée comme dict `VALID_TRANSITIONS`.

5.1 États

État	Sémantique
IDLE	Aucune session active (jamais démarrée ou cleanup en cours)
INDEXING	Ingestion d'un cours en cours (bloqué côté UX)
PRESENTING	TTS en train de jouer une narration
LISTENING	VAD actif, attente d'audio étudiant
PROCESSING	STT + RAG + LLM en cours (Q&A)
RESPONDING	TTS de la réponse Q&A
WAITING	Pause utilisateur — attente d'un trigger de resume
CLARIFICATION	Étudiant demande clarification / revenir slide

5.2 Transitions valides

From → To	IDLE	INDEXING	PRESENTING	LISTENING	PROCESSING	RESPONDING	WAITING	CLARIFICATION
IDLE		■	■	■				
INDEXING	■		■					
PRESENTING				■			■	■
LISTENING			■		■			■
PROCESSING	■		■	■		■		■
RESPONDING			■	■			■	■
WAITING	■		■	■	■			■
CLARIFICATION			■	■		■		

5.3 SessionContext (Redis)

```

@dataclass
class SessionContext:
    session_id: str
    state: str # DialogState.value
    language: str = "fr"
    student_level: str = "lycée"
    course_id: Optional[str]
    chapter_index: int = 0
    section_index: int = 0
    char_position: int = 0 # legacy char-based resume
    last_narrated_idea_id: str = "" # idea-based resume cleaner
    student_id: str = "" # JWT sub
    course_summary: str = ""
    course_analysis: dict = {}
    last_slide_explained: str = ""
    last_concept_explained: str = ""
    last_narration_summary: str = ""
    confusion_count: int = 0
    last_question_hash: str = ""
    repeated_question_count: int = 0

```

```

    last_confusion_score: float = 0.0 # EWMA fused score
    consecutive_clean_turns: int = 0
    interruptions: int = 0
    student_baseline: dict = { # Couche #1 adaptive
        "avg_speech_rate": 120.0, # wpm baseline
        "avg_question_length": 8,
        "avg_questions_per_turn": 1.2,
        "hesitation_baseline": 1.0,
        "confusion_threshold_multiplier": 1.0,
        "turns_analyzed": 0,
    }
    paused_state: dict = {
        "is_paused": False,
        "slide_id": None,
        "char_offset": 0,
        "last_idea_id": "",
        "timestamp": 0.0,
        "presentation_text": "",
        "presentation_cursor": 0,
        "presentation_key": "",
        "slide_title": "",
        "slide_path": "", # PNG pour Q&A vision
        "slide_content": "", # OCR text pour Q&A
        "presentation_text_len": 0,
    }
    last_activity: float = ...

```

5.4 Logs FSM

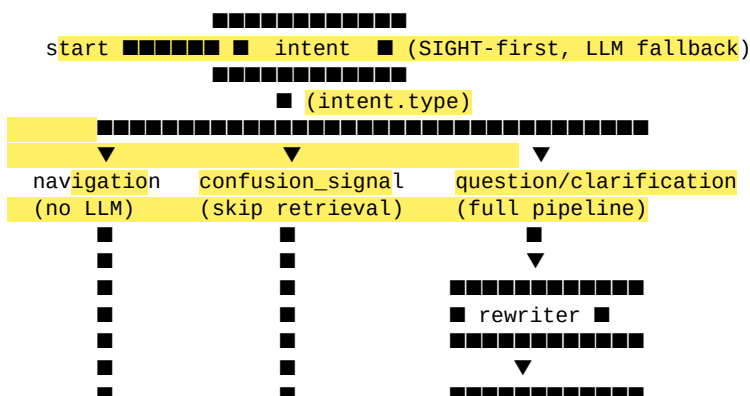
- [b55337f0] IDLE → LISTENING
- [b55337f0] LISTENING → PRESENTING
- [b55337f0] PRESENTING → LISTENING
- [b55337f0] WAITING → PROCESSING
- [b55337f0] PROCESSING → RESPONDING
- [b55337f0] RESPONDING → LISTENING
- [b55337f0] Transition invalide : IDLE → RESPONDING

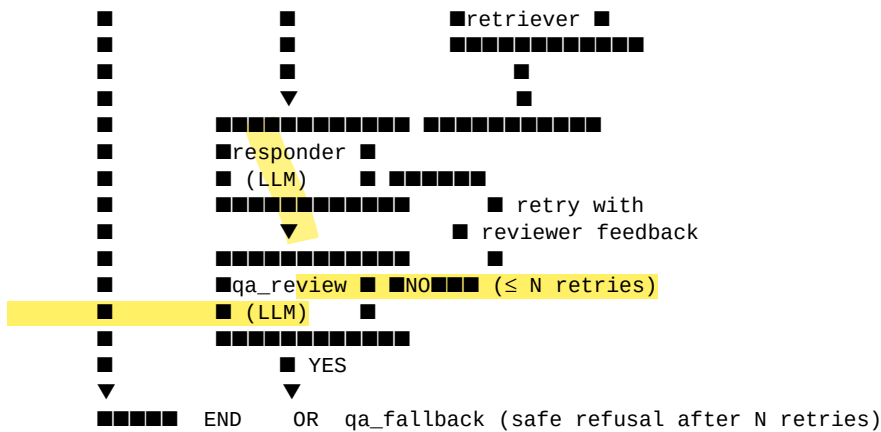
PARTIE II — PIPELINES

6. Q&A Graph

Module : `agentic/qa/graph.py`. Compose 5 nœuds + retry loop.

6.1 Diagramme





6.2 Intent Agent

Source : [agentic/qa/intent.py](#).

Étape 1 — SIGHT classifier :

- Modèle XLM-RoBERTa-base fine-tuné (file `dataset/sight-main/.../confusion_model_final.pth`)
- Tokenize question (max 96 tokens)
- Forward pass sur CPU (~25ms-3s selon premier load)
- Output : sigmoid → `prob ∈ [0, 1]`
- Si `prob ≥ 0.6` (threshold) → `intent = confusion_signal` (skip LLM)

Étape 2 — LLM intent classifier (si SIGHT pas confiant) :

Prompt : Tu es un classificateur d'intent pour un tuteur IA. Question : "{q}"

Réponds JSON strict :

```
{ "intent": "question|clarification|confusion_signal|navigation|off_topic",
  "confidence": 0.1,
  "needs_retrieval": true|false,
  "is_definition": true|false,
  "definition_term": "...",
  "anchored_concept": "...",
  "needs_rewrite": true|false,
  "rewritten": "..."}

```

LLM call routed via `LLMRouter.invoke(prompt, prefer="openai")` :

- 1er essai : OpenAI gpt-4o-mini
- Fallback : Groq llama-3.3-70b
- Fallback : Ollama Mistral

Logs visibles :

```

■ intent USER text : 'qu est ce que la RI ?'
intent: SIGHT-classified as confusion_signal (prob=0.73)
(OR si LLM)
■ intent LLM raw : '{"intent":"question","confidence":0.95,...}'
intent: question (conf=0.95, retrieve=True) | rewriter merged: yes (24→18, 'Qu'est-ce que la RI ?')

```

6.3 Rewriter Agent

Source : [agentic/qa/rewriter.py](#).

Si `intent.needs_rewrite=true` et que le rewrite n'a pas déjà été fait par l'intent LLM, ré-écrit la query pour amplifier le rappel RAG.

Exemple : `"ri ?"` → `"Qu'est-ce que la recherche d'information ?"`.

Prompt typique :

Réécrit cette question d'étudiant pour qu'elle soit explicite et complète, en ajoutant le contexte du chapitre actuel (`{chapter_title}`).
Question originale : `"{question}"`
Réécriture (1 phrase, max 20 mots) :

Skip si rewriter déjà fait par intent → log : `rewriter: skipped (merged with intent, rewrite=18 chars)`.

6.4 Retriever Agent

Source : `agentic/qa/retriever.py`. Délègue à `MultiModalRAG.retrieve_chunks`.

Pipeline détaillé :

1. Vector search (`_vector_search`) :

- Encode query avec BGE-m3 → vecteur 1024-dim
- Qdrant `similarity_search_with_score` avec filter `course=course_id` (+ optionally `chapter_idx`)
- Top $K \times 3$ résultats par cosine similarity

2. BM25 search (`_bm25_search`) :

- In-memory BM25Retriever (langchain) construit à l'ingestion
- Top $K \times 3$ résultats par BM25 score
- Filter manuel sur `metadata.course == course_id`

3. Compute α adaptatif (`_bm25_alpha`) : `avg_idf = mean(idf(term) for term in query_tokens)`
`normalized = (avg_idf - lo) / (hi - lo) clamped to [0, 1]` $\alpha = 1.0 + 0.32 \times \text{normalized} \in [1.0, 1.32]$ Termes rares → α haut → boost BM25 (matches exacts).

4. RRF fusion : `score(d) = \alpha / (60 + rank_bm25(d)) + (2 - \alpha) / (60 + rank_dense(d))` $k=60$ constant standard (Cormack 2009).

5. Cross-encoder rerank (si `RAG_USE_RERANKER=true`) :

- Top `RAG_RERANKER_TOP_N=15` candidats issus de RRF
- Pour chaque (query, chunk_text) → bge-reranker-v2-m3 → score sigmoid([0, 1])
- Re-tri par `rerank_score` desc

6. Top-K final : `RAG_NUM_RESULTS=5` chunks retournés.

7. KG-augmentation optionnelle (`RAG_USE_GRAPH_EXPANSION=true`) :

- Top-1 chunk → suit edges `depends_on_ids` et `illustrates_id`
- Ajoute prereqs / exemples / le concept illustré dans la liste finale

Logs visibles :

```

■ Retrieval | ch=1 | strict=True | course=9940c8f4 | q='qu est ce que la RI ?'
■ QDRANT search START | collection=smart_teacher_multimodal... k=15 | course=9940c8f4 chapter=1 | filters=2 |
■ QDRANT search DONE | hits=15/15 | top_score=0.847 mean=0.692 | took=42ms
  ■■ BM25 boost  $\alpha=1.32$  (query has rare terms)
  ■ 5 chunks retenus (rerank: 0.62 [cross-encoder], cosine: n/a [BM25 only])
  ■ rag[0] rerank=0.731 cosine=n/a ch=1 sec=10 src=... | "La RI est ..."
...
retriever: 5 direct + 0 kg-augmented (course=9940c8f4 ch=1, seen=0, review_mode=False)

```

6.5 Responder Agent

Source : `agentic/qa/responder.py` (~1 800 lignes).

Construction du prompt (mode `question`, FR) :

```

<<INSTRUCTION_INTERNE_NE_PAS_MENTIONNER_DANS_LA_REPONSE>>
Style cognitif à respecter : {style-specific instructions, e.g.
  "Privilégie des analogies visuelles, des schémas mentaux..."}
Format ÉQUILIBRÉ : 3-4 phrases.
Ton {tone} : ...
<<FIN_INSTRUCTION_INTERNE>>

<<INSTRUCTION_INTERNE_NE_PAS_MENTIONNER_DANS_LA_REPONSE>>
Stratégie pédagogique à appliquer : {bandit reasoning, e.g.
  "Adopte un style SOCRATIQUE : pose 1 ou 2 questions guidantes..."}
<<FIN_INSTRUCTION_INTERNE>>

```

Tu es Smart Teacher, un tuteur IA qui répond à la question d'un étudiant en cours.

■■■ RÈGLE – TUTEUR LIMITÉ AU MATÉRIEL DU COURS : tu réponds à partir de TOUT le matériel du cours fourni ci-dessous (la SLIDE EN COURS et le CONTEXTE INTERNE – qui peut contenir des extraits de N'IMPORTE QUELLE partie du cours, pas seulement de la slide en cours). L'étudiant peut poser des questions qui dépassent la slide affichée – c'est légitime. Tu n'es PAS un chatbot encyclopédique : tu n'utilises PAS ta connaissance générale, Wikipédia, ni d'exemples extérieurs au cours. Si la question porte sur un sujet qui n'est ni dans la slide ni dans le contexte interne, tu DOIS répondre honnêtement par UNE SEULE phrase, ex : "Ce point n'est pas abordé dans ce cours – je ne peux pas l'expliquer ici sans m'écarter du programme." (et laisse "supporting_chunks" vide).

INSTRUCTION PÉDAGOGIQUE : 5-7 phrases naturelles. Structure obligatoire :

1. DÉFINITION précise du concept (1-2 phrases)
2. REFORMULATION en mots plus simples
3. INTUITION : à quoi ça sert
4. UN EXEMPLE CONCRET tiré du cours
5. (Optionnel) Lien avec un concept proche

ANCRAGE : la SLIDE EN COURS est ta source de vérité prioritaire. Le CONTEXTE INTERNE est un complément ; chaque morceau y est étiqueté par [id:xxx]. Certains morceaux portent un label [id:xxx | prerequisite] (à connaître AVANT) ou [id:xxx | exemple concret].

Le champ "answer" est lu à voix haute : pas de markdown, pas de [id:...] dans le texte parlé – les ids vont uniquement dans "supporting_chunks".

■■■ SLIDE EN COURS ■■■
{slide_content}

■■■ CONTEXTE INTERNE (chaque morceau a un [id:xxx]) ■■■
{chunks_block_with_ids}

■■■ ÉCHANGES RÉCENTS (continuité – ne les répète pas) ■■■
{history_text}

■■■ QUESTION ACTUELLE ■■■
{question}

Réponds UNIQUEMENT en JSON strict, sans markdown :
{ "answer": "...", "supporting_chunks": ["id1", "id2"] }

Mode confusion_signal : pas de chunks → prompt de reformulation simplifié.

Mode navigation : skip LLM, emit Action navigate avec sub-action (next/prev/repeat/skip/...).

6.6 Reviewer Agent (Self-RAG)

Source : `agentic/qa/reviewer.py`. Vérifie le grounding.

Prompt :

La réponse suivante est-elle GROUNDÉE dans la slide + chunks fournis ?

"Groundée" = chaque affirmation factuelle vient du contenu (pas inventée, pas Wikipédia).

Slide : {slide_text}

Chunks : {chunks_text}

Question : {question}

Réponse : {answer}

JSON strict : {"grounded": true|false, "feedback": "..."}
Si grounded: false ET retries restants (max 2 par défaut) :

- Re-call répondre avec le feedback ajouté en suffixe : ■■ TENTATIVE PRÉCÉDENTE REJETÉE – raison : {feedback}. Corrige ce point précis.

Si max retries atteint → qa_fallback emit la phrase de refus :

- FR : « Je ne suis pas sûr de pouvoir répondre précisément à cette question à partir du cours. Pouvez-vous reformuler ou préciser ? »
- EN : « I'm not sure I can answer this from the course material. Can you rephrase or clarify? »

6.7 Off-topic guardrail

Source : `agentic/qa/responder.py_detect_leak_or_offtopic`.

Calcul du lexical overlap : 1. Tokenize answer + (slide ∪ chunks) 2. Filtre stop-words (FR : le, la, les, un, une, ...; EN : the, a, an, ...) 3. Lowercase + strip ponctuation 4. $overlap = |answer \cap source| / |answer|$ 5. Reject si :

- $overlap < Config.GROUNDING_OVERLAP_THRESHOLD = 0.08$ AND
- Pas de citation supporting_chunks

Si reject → remplace par le safe refusal.

Log :

■ guardrail | overlap=0.89 (threshold=0.08) | answer_words=19 source_words=51 | common=17 | has_citations=False

6.8 Q&A graph timings (typical)

Avec Groq :

Étape	Latence	Notes
intent (SIGHT only)	25-100ms	classifier local
intent (LLM)	0.5-1.5s	Groq

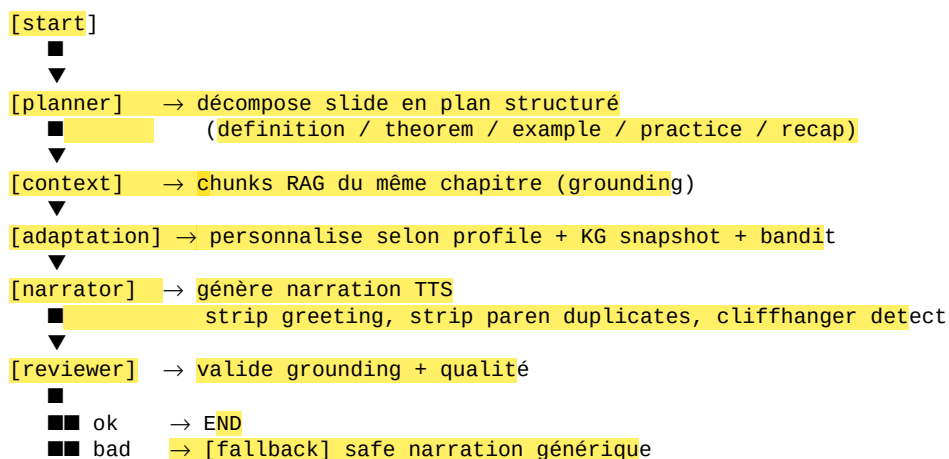
Étape	Latence	Notes
rewriter	0 (skipped)	merged with intent
retriever	0.8-2.5s	Qdrant + BM25 + rerank
responder	0.8-2.0s	Groq
guardrail	<10ms	local
qa_review	0.3-1.0s	Groq
Total	2-7s	end-to-end

Avec Ollama (CPU only) : 60-600s (impraticable, ne pas utiliser pour Q&A interactif).

7. Teaching Graph

Module : `agentic/teaching/graph.py`. Compose 6 nœuds.

7.1 Diagramme



7.2 Planner

LLM décompose le contenu de la slide en `plan: list[PlanStep]`.

```

@dataclass
class PlanStep:
    type: str # "definition", "example", "practice", "recap"
    content: str # mini-narration de 1-2 phrases
    depth: str # "basic", "balanced", "deep"
    concept: str # main concept identifier
    chunk_ids: list[str] # supporting RAG chunks

```

Prompt simplifié :

Tu es un planificateur de cours pédagogique. Décompose cette slide en étapes naturelles d'enseignement. Pour chaque étape, donne le type, contenu, depth.
Slide : {slide_text}
Chapitre : {chapter_title}
Niveau : {student_level}

JSON : {"plan": [{"type": "definition", "content": "...", "depth": "basic"}, ...]}

7.3 Context

Pour chaque **PlanStep**, recherche RAG dans le chapitre courant :

```
chunks = rag.retrieve_chunks(  
    query=step.content,  
    k=3,  
    course_id=course_id,  
    current_chapter_idx=chapter_idx,  
    strict_chapter=True,  
)  
step.chunk_ids = [c.metadata["idea_id"] for c in chunks]
```

7.4 Adaptation

Combine 3 signaux : 1. **Profile** : `style ∈ {visual, verbal, kinesthetic}, pace, depth, tone` 2. **Knowledge snapshot** : `strong / weak / never_seen / recently_confused concepts` 3. **Bandit decision** : `strategy + speech_rate`

Modifie le plan en place :

- Si `concept ∈ strong_concepts` → réduit la depth (don't redefine)
- Si `concept ∈ weak_concepts` → ajoute un exemple
- Si `concept ∈ recently_confused` → simplifie le vocabulaire
- Si `concept ∈ never_seen_concepts` → ajoute la définition complète

Logs visibles :

```
■ adaptation_level=lycée confused_topics=[] bumps=socratic  
■ adapted_plan[0] type=definition depth=basic concept=k_means  
■ adapted_plan[1] type=example depth=concrete concept=k_means
```

7.5 Narrator

Génère le texte final de la narration. Prompt (FR) :

`{personalization_prefix}`

Tu es SMART TEACHER, un tuteur IA spécialisé en `{domain_name}`, qui présente un cours à des `{audience_fr}`.
Tu expliques le contenu de la slide en tant que pédagogue qui commente le matériel, PAS en tant qu'auteur humain des slides.

RÈGLES D'IDENTITÉ :

- NE JAMAIS dire 'Je suis [nom]', 'Bienvenue à [Cours]'.
- Si la slide mentionne un nom d'enseignant, NE LE RÉPÈTE PAS.
- N'ouvre PAS chaque slide par 'Bienvenue' ou 'Aujourd'hui nous allons'.

RÈGLES TERMINOLOGIQUES :

- Conserve les acronymes EXACTEMENT comme la slide les écrit.
- NE PAS inventer d'expansion.

RÈGLES DE PRÉSENTATION :

- 4 à 6 phrases d'EXPLICATION CONCRÈTE.
- INTERDIT : NE TERMINE JAMAIS par une question rhétorique ouverte.
- OBLIGATOIRE : si la slide cite un concept, donne une explication ou exemple.

■■■ SLIDE EN COURS ■■■
`{slide_text}`

■■■ CONTEXTE INTERNE ■■■
`{chunks_text}`

■■■ PLAN À SUIVRE ■■■

{adapted_plan}

Post-processeurs sur le texte généré : 1. `_strip_greeting_opener` — supprime "Bonjour", "Bienvenue dans..." 2. `_strip_parenthetical_duplicates` — supprime "X (X)" → "X" 3. `_is_cliffhanger` — détecte teaser sans payoff ("...que nous allons définir." à la fin) 4. `_clean_for_speech` — strip markdown / LaTeX

Logs :

- narrator AUGMENTED PROMPT (chars=2487 turns=0)
- narrator LLM RAW (670 chars): "..."
- stripped greeting (24 chars)
- stripped paren duplicates (12 chars)

7.6 Reviewer (teaching)

Identique à QA reviewer mais évalue la fidélité de la narration au plan + slide.

7.7 Fallback

Si reviewer rejette + retries épuisés → narration générique :

"Cette slide aborde le concept de {main_concept}. Je vais vous laisser la lire et n'hésitez pas si vous avez des questions."

8. RAG multimodal

Module : `rag/multimodal_rag.py` (~3 000 lignes).

8.1 Pipeline d'ingestion (étape par étape)

PDF upload (POST /course/build)

- - ▼
- [1] `IntelligentIngester.ingest_file`
 - • `pypdf` → texte brut par page
 - • `pdf2image` → PNG 200dpi par page
 - • `pillow` + `pytesseract` → OCR images embedded
 - • `unstructured` → Tables, FigureCaptions, Titles
 - Output : `IngestionResult{pages[], assets[], language, ...}`
 - - ▼
- [2] `CourseBuilder.build_from_file`
 - • Détection domaine/cours via LLM ou heuristique
 - • Pour chaque page : compute section title via vision LLM ou structural
 - • Group pages → chapters/sections
 - • Save to Postgres : courses, chapters, sections, ingested_asset
 - Output : (course_id_uuid, course_data dict)
 - - ▼
- [3] `rag.run_ingestion_pipeline_from_course_data(course_data, course_id=uuid)`
 - - `_documents_from_course_data`
 - ■ Pour chaque section :
 - ■ • Si `RAG_USE_IDEA_CHUNKING=false` (default) :
■ ■ `ideas = [{idea: section.content, label: "fragment"}]`
 - ■ • Si true :
■ ■ `ideas = _chunk_by_ideas(section.content, ...)`
■ ■ → LLM segmente en {definition, theorem, example, ...}

```

■ ■ → DANGER : Ollama hallucine "RI = Research Interface"
■ ■ Pour chaque idea :
■ ■   • idea_id = md5(course|chap|sec|i|j|content_hash)[:12]
■ ■   • Build Document(page_content=idea_text, metadata={...})
■ ■ Output : list[Document] (typiquement 50-300 docs par cours)
■
■ ■ ■ _store_documents_once
■ ■   • Encode tous les docs avec BGE-m3 (batch, GPU si dispo)
■ ■   • qdrant.create_collection(...) si manquant
■ ■   • qdrant.upsert(points=[(id, vec, payload), ...])
■ ■   • Build BM25 retriever in-memory
■ ■   • Save docs_cache.json on disk (cross-restart cache)
■
■ ■ ■ KG build
■ ■   • Pour chaque idea avec depends_on_ids → IdeaNode
■ ■   • ConceptFromTitles.extract(rag.all_docs, course_id)
■ ■       → group by section_title → ConceptInfo per group
■ ■       → optional LLM enrich (canonical, description, bloom, sub_concepts)
■ ■   • kg.attach_concepts(concepts)
■
■ ■ ■ Persist KG concepts to disk + DB

```

8.2 Pipeline de retrieval (étape par étape)

```

retrieve_chunks(query, k=5, course_id, current_chapter_idx, strict_chapter=False)
▼
[1] expand_query (light query expansion based on language)
■ ■ Exemple : "ri" → "ri recherche d'information"
▼
[2] _vector_search(query, k=k*3, filter=course+chapter)
■ ■   • Encode query → 1024-dim vec (BGE-m3)
■ ■   • qdrant.similarity_search_with_score (Cosine)
■ ■   • Top k*3 candidats
■ ■ Output : list[(doc, cosine_score)]
▼
[3] _bm25_search(query, k=k*3, filter=course)
■ ■   • BM25Retriever.invoke(query)
■ ■   • Manual filter on metadata.course
■ ■   • Top k*3
■ ■ Output : list[doc]
▼
[4] _bm25_alpha(query) →  $\alpha$ 
■ ■   • Pour chaque token, lookup IDF
■ ■   • avg_idf = mean(idf)
■ ■   • normalize to [0, 1] via thresholds
■ ■   •  $\alpha = 1.0 + 0.32 \times \text{normalized}$ 
▼
[5] _rrf_fuse(vector_docs, bm25_docs, alpha= $\alpha$ )
■ ■   • Pour chaque doc d :
■ ■       score(d) =  $\alpha / (60 + \text{rank\_bm25}(d)) + (2 - \alpha) / (60 + \text{rank\_dense}(d))$ 
■ ■   • Sort desc by score
■ ■   • Return top RAG_RERANKER_TOP_N=15
▼
[6] _rerank_with_cross_encoder(query, top15)
■ ■   • Pour chaque (query, chunk_text) → bge-reranker-v2-m3 → score
■ ■   • Sort desc
▼
[7] _augment_with_kg (optional)
■ ■   • top1.idea_id → suit edges depends_on_ids, illustrates_id
■ ■   • Prepend prereqs/examples to result
▼
[8] Return top-k=5 chunks_with_scores

```

8.3 Schéma metadata par chunk Qdrant

```
{
  "source_file": "courses/.../Chapitre 1.pdf",
  "chunk_idx": 0,
  "idea_index_in_chunk": 0,
  "domain": "informatique",
  "course": "9940c8f4-2392-4158-9126-20c1a619b83d",
  "language": "fr",
  "original_text": "La RI est le processus...",
  "content_hash": "ab12cd34",
  "chapter_idx": 1,
  "chapter_title": "Chapitre 1: Introduction A La Ri",
  "section_idx": 3,
  "section_title": "II. Définitions et terminologie",
  "idea_id": "f234bf423c96",
  "idea_label": "definition",
  "slide_idx": 3,
  "image_url": "/media/slides/.../page_003.png",
  "depends_on_ids": ["abc12def3456"],
  "illustrates_id": null
}
```

8.4 Vision describe (services/vision_describe.py)

Pour chaque slide PNG, un seul appel LLM vision (gpt-4o-mini-vision OU Ollama llava), résultat caché sur disque par `md5(image_bytes).hexdigest()` dans `cache/vision_descriptions/{md5}.json`.

Structure :

```
{
  "description": "Schéma d'un index inversé : 3 colonnes (terme, doc_id, fréquence)...",
  "language": "fr",
  "main_concept": "Index inversé",
  "captured_at": 1746000000.0,
  "model": "gpt-4o-mini"
}
```

Au moment du Q&A, `merge_into_slide_content` combine OCR text + vision description :

```
{ocr_text}
```

```
[Description visuelle] : {vision.description}
```

9. Knowledge Graph

Module : `pedagogy/knowledge_graph/`.

9.1 Data classes

```
@dataclass
class IdeaNode:
    idea_id: str # md5 hash, primary key
    label: str # definition | theorem | example | warning | note | fragment
    text: str
    course_id: str
    chapter_idx: int = 0
    section_idx: int = 0
    depends_on_ids: set[str] = field(default_factory=set)
    illustrates_id: Optional[str] = None

@dataclass
```

```

class ConceptInfo:
    name: str # snake_case "k_means"
    display_name: str = ""
    canonical_name: str = ""
    description: str = ""
    bloom_level: str = "" # remember|understand|apply|analyze|evaluate|create
    course_id: str = ""
    chapter_idxs: set[int] = field(default_factory=set)
    score: float = 0.0
    idea_ids: set[str] = field(default_factory=set)

```

9.2 Edges dérivés

Edge idea-level	Sémantique	Source
A --depends_on--> B	A nécessite la compréhension de B	LLM idea-chunking
A --illustrates--> B	A est un exemple de B	LLM idea-chunking

Edge concept-level	Définition
C1 --prereq--> C2	$\exists \text{ idea } i1 \in C1, i2 \in C2 / i1 \text{ depends_on } i2 \text{ (et } C1 \neq C2)$
C1 --example--> C2	$\exists \text{ idea } i1 \in C1, i2 \in C2 / i1 \text{ illustrates } i2 \text{ (et } C1 \neq C2)$

9.3 Construction des concepts (ConceptFromTitles)

5 étapes, toutes déterministes sauf l'étape 4 (LLM enrich) :

1. Group docs par `section_title` (TOC du PDF) — déterministe ■ 2. Filter noise (`{intro, summary, agenda, ...}`, `len < 3`, `isdigit`) — déterministe ■ 3. Merge titres similaires (canonical lowercase) — déterministe ■ 4. `_llm_enrich(title, excerpts)` — LLM, peut halluciner ■■

- Si `Config.KG_DISABLE_LLM_ENRICH=true` (default) → skip → utilise titre brut
- Sub-concepts validés par substring match dans le texte ; non-trouvés sont **droppés** (anti-halluc)

5. Tri par `len(chunk_ids)` desc + cap `max_concepts=50`

9.4 Queries disponibles

```

# Idea-level
get(idea_id) → IdeaNode
prerequisites_of(idea_id, transitive=False) → list[IdeaNode]
dependents_of(idea_id, transitive=False) → list[IdeaNode]
examples_of(idea_id) → list[IdeaNode]
learning_path(target_id, mastered: set[str]) → list[IdeaNode] # topological order

# Concept-level
list_concepts(course_id=None) → list[ConceptInfo]
get_concept(name) → Optional[ConceptInfo]
ideas_in_concept(name) → list[IdeaNode]
concepts_of_idea(idea_id) → list[ConceptInfo]
prereq_concepts(name) → list[ConceptInfo]
example_concepts_of(name) → list[ConceptInfo]
count_idea_edges_between(source_name, target_name) → int # epaisseur visuelle Cytoscape

```

9.5 Stats typiques

Cours « Recherche d'Information » (chapitre 1, 19 sections) :

```
{
  "nodes": 64,
  "depends_edges": 51,
  "illustrates_edges": 1,
  "courses": 1,
  "roots": 23, # no prerequisites
  "leaves": 33, # no dependent
  "avg_in_degree": 0.8,
  "concepts": 14,
  "concept_links": 64
}
```

PARTIE III — PÉDAGOGIE

10. Personnalisation

10.1 Student profile (Redis)

Clé : `profile:{student_id}:{course_id}` ou `profile:{student_id}:_global`.

Champs typiques :

```
{
  "student_id": "ce566ac0-...",
  "course_id": "9940c8f4-...",
  "language": "fr",
  "level": "lycée",
  "style": "visual",
  "pace": "normal",
  "depth": "balanced",
  "tone": "challenging",
  "preferences": {
    "speech_rate": 0.95,
    "manual_rate_override": false,
    "pitch": 1.0,
    "voice": "default",
    "vark_visual": 0.45,
    "vark_verbal": 0.30,
    "vark_kinesthetic": 0.25
  },
  "learning_style_posterior": {
    "visual": 0.45,
    "verbal": 0.30,
    "kinesthetic": 0.25
  },
  "avg_response_time_s": 12.5,
  "confusion_rate": 0.17,
  "preferred_explanation_depth": "balanced",
  "turns_analyzed": 42,
  "last_updated": 1746000000.0
}
```

10.2 VARK learning style (Bayes posterior)

Source : `pedagogy/personalization/learning_style/bayes.py`.

Prior uniforme `Dirichlet(1, 1, 1, 1)` sur (visual, auditory, read-write, kinesthetic). Update à chaque interaction signifiante :

- Étudiant pose une question avec « voir », « image », « schéma » → +visual
- Étudiant écoute attentivement (peu d'interrupts) → +auditory
- Étudiant prend des notes textuelles → +read-write
- Étudiant fait un exercice → +kinesthetic

Posterior : `Dirichlet( $\alpha_v + n_v, \dots$ )` → moyenne $\alpha_i / \text{sum}(\alpha_j)$.

10.3 Bandit Thompson sampling

Source : `pedagogy/personalization/bandit/thompson.py`.

Arms = (strategy × speech_rate) :

- strategies = {socratic, simpler_words, decomposition, analogy, recap, example}
- speech_rates = {slow, normal, fast}
- → 18 arms

Posterior par arm : Beta(α, β) avec prior uniforme Beta(1, 1).

Context buckets (feature hashing) :

- confusion_bucket : low / medium / high (selon last_confusion_score)
- mastery_bucket : low / medium / high (selon avg mastery dans le snapshot)
- engagement_bucket : disengaged / neutral / engaged
- interaction_bucket : isolated / mixed / dense (selon turn rate)

→ Hash en string "`{conf}|{mast}|{eng}|{interact}`", ex "`mixed|fast|medium|isolated`".

Chaque bucket maintient son propre dict d'arms (warm start cross-bucket via global mean).

Decision :

```
samples = {arm: Beta( $\alpha$ [arm],  $\beta$ [arm]).rvs() for arm in arms}
chosen = max(samples, key=samples.get)
```

Update après le turn :

```
reward = compute_reward(outcome) #  $\in [0, 1]$ 
 $\alpha$ [chosen] += reward
 $\beta$ [chosen] += (1 - reward)
```

Persisté en Postgres (`bandit_posterior`s table : course_id, bucket, arm, alpha, beta, n_pulls, last_pull_at).

Logs :

```
■ bandit THOMPSON | bucket=mixed|fast|medium|isolated | 18 arms | winner=socratic:slow draw=0.997
■ arm=socratic:slow       $\alpha$ =1  $\beta$ =1 mean=0.50 sample=0.997 ← chosen
■ arm=simpler_words:fast  $\alpha$ =1  $\beta$ =1 mean=0.50 sample=0.966
■ arm=decomposition:fast  $\alpha$ =1  $\beta$ =1 mean=0.50 sample=0.899
■ arm=socratic:normal     $\alpha$ =1  $\beta$ =1 mean=0.50 sample=0.829
■ arm=simpler_words:normal  $\alpha$ =1  $\beta$ =1 mean=0.50 sample=0.794
■ bandit START | session=b55337f0 ctx_bucket=mixed|fast|medium|isolated | strategy=socratic speech_rate=slow |
```

11. Pause / Resume intelligent

Module : `pedagogy/resume_intelligence.py`.

11.1 Six intents

Intent	Condition	Stratégie
QUICK_RESUME	pause < 10s, mid-narration	CONTINUE
NORMAL_RESUME	$10s \leq \text{pause} < 60s$	REWIND_SENTENCE
RESUME_AFTER_GAP	$60s \leq \text{pause} < 180s$	REEXPLAIN_AND_CONTINUE
RESUME_AFTER_LONG	pause $\geq 180s$	REEXPLAIN_AND_CONTINUE
SLIDE_COMPLETED	cursor $\geq$ end, pause < 30s	SKIP_TO_NEXT
REVIEW_REQUEST	cursor $\geq$ end, pause $\geq 30s$	REWIND_SENTENCE

11.2 Cinq stratégies

Stratégie	Effet
CONTINUE	keep cursor, no recap, jouer depuis cursor
REWIND_SENTENCE	rewind to start of current sentence (search backward <code>[.!?...]+\s+</code>), pas de recap
REWIND_SENTENCE_RECAP	rewind + medium recap (1-2 phrases)
REWIND_SENTENCE_LONG_RECAP	rewind + long recap (3-4 phrases)
REEXPLAIN_AND_CONTINUE	Appel LLM pour ré-expliquer la phrase courante en mots différents, puis play le reste cached
SKIP_TO_NEXT	signal frontend "slide done, advance"

11.3 Cursor preservation au pause

Frontend envoie `audio_progress` $\in [0, 1]$ lors d'un interrupt :

```
cursor = round(audio_progress * len(narration))
cursor = max(0, min(cursor, len(narration)))
```

Stocké dans `paused_state.presentation_cursor` + Redis snapshot.

11.4 Revisit-completed-slide guard

Quand l'étudiant revient sur une slide finie :

```
end_threshold = int(len(snapshot_text) * 0.95)
if paused_cursor is None and snap_cursor >= end_threshold:
    cursor = 0 # replay from start
```

Sinon le TTS reprendrait à la fin de la narration → silence → impression de skip.

11.5 Live waiting ticker

Pendant la pause, un asyncio Task logue :

- Intervals fixes : [15, 30, 60, 90, 120, 180, 240, 300, 420, 600, 900, 1200, 1800, 2400, 3000, 3600] s
- Au-delà : tick toutes les `TAIL_STEP_S = 1800` s (30 min)

Au resume :

■ WAITING ticker stopped | total_wait=42.3s (0m42s) | reason=pause

11.6 Logs flux complet pause→resume

```
■ TTS GENERATE DONE | took=2.45s | speech_speed=27ch/s
■ Audio streamed: 145200 bytes
... (étudiant écoute) ...
■ interrupt msg | audio_progress=0.7035 reason='pause' current_cursor=670 current_text_len=670
■ audio_progress→cursor | progress=0.7035 × narration_len=670 = cursor=471 (70.4% played)
■ Interrupt cursor override 670 → 471 (audio_progress=0.70, narration_len=670)
■ PAUSE START | reason=pause | slide=...:0:0 | cursor=471/670 (70.3%) | timestamp=2026-05-04T...
■ REDIS SETEX snapshot | text_len=670 cursor=471 | took=4.2ms
■ PAUSE_SESSION | cursor=471/670 (70.3%) | timestamp=... | interruptions_total=1
■ PAUSE STATE SAVED | took=8ms
■ WAITING ticker STARTED | reason=pause slide=...:0:0
■ interrupt latency = 87ms

■ WAITING tick | elapsed=15s (0m15s) | bucket=NORMAL
■ WAITING tick | elapsed=30s (0m30s) | bucket=NORMAL
... (étudiant absent 42s) ...

■ RESUME TRIGGERED | trigger='reprendre' | wait_total=42.3s (0m42s)
■ REDIS GET HIT | key=session:... took=0.6ms
■ RESUME_SESSION | cursor=471/670 (70.3%) | pause_duration=42.3s
■ detect_resume_intent | cursor=471/670 (70.3%) pause=42.3s
■ pause_bucket | duration=42.3s < 60s (NORMAL) → 'normal'
■ detect_resume_intent → normal_resume (bucket=normal)
■ compose_resume_action FINAL | strategy=rewind_sentence
■ rewind_to_current_sentence_start | cursor=471 → start=318 (skipped back 153 chars across 2 boundaries)
■ cache_decide HIT memory + paused | text_len=670 cursor=471 (70.3%) | resume from paused position
■ Audio streamed (cached) ...
```

12. Détection de confusion

Module : `pedagogy/confusion/`.

12.1 SIGHT model

XL_M-RoBERTa-base fine-tuné (Bhatti 2022, dataset SIGHT) :

- Input : tokenized question (max_length=96)
- Output : sigmoid → `prob_confused` ∈ [0, 1]
- Threshold : 0.6
- Device : CPU (~25ms after warm load, 3-30s first load due to weights download)

Path : `dataset/sight-main/data/processed/confusion_model_final.pth`.

12.2 Prosody markers (heuristiques)

Source : [audio/transcriber.extract_prosody.](#)

Output: {"speech_rate": 145, "hesitation_count": 2, "markers": [...]} avec markers ∈ :

- `slow_speech_rate` : `wpm < 100`
- `frequent_hesitations` : `"euh|hum|...|eh" ≥ 2`
- `high_silence_ratio` : `silence_duration / audio_duration > 0.4`

12.3 Fusion (formule)

Source : [pedagogy/confusion/fusion.py](#).

$$\begin{aligned}\text{fused_score} &= W_{\text{TEXT}} \times \text{sight_score} + W_{\text{PROSODY}} \times \text{prosody_score} \\ &= 0.7 \times \text{sight} + 0.3 \times \text{prosody}\end{aligned}$$

prosody_score from markers count :

markers	prosody_score
0	0.00
1	0.30
2	0.70
3	1.00

Single signal handling :

- Sight only, no prosody → fused = sight (no penalty)
- Prosody only, no sight → fused = prosody

Decision : $\text{is_confused} = \text{fused} \geq 0.5$.

12.4 Confusion persistence

Source : pedagogy/confusion/persistence.py.

À chaque détection : INSERT dans `confusion_events` (Postgres):

```
INSERT INTO confusion_events
(id, student_id, course_id, session_id, idea_id, score, reason, prosody_dict, sight_score, prosody_score, cre
VALUES (...);
```

Aggregations utilisées par engagement scorer + bandit reward.

13. Adaptation du débit de parole

Module : `pedagogy/personalization/tts_adapter.py`.

13.1 Architecture (3 couches + override)



13.5 Override manuel utilisateur

L'étudiant POST `/session/{id}/speech_rate` :

```
{"rate": 0.85, "manual_override": true}
```

Quand `manual_override=true` :

- Bandit modulation IGNORÉE
- Confusion slowdown IGNORÉ
- Le rate stocké est celui utilisé directement

Pour ré-activer l'auto :

```
{"manual_override": false}
```

13.6 Logs visibles

Auto :

```
■■ tts rate composed | profile=1.00 (level_default(lycée)) × bandit=1.15 (fast) × confusion=1.00 (score=0.10) =
```

Override :

```
■■ tts rate USER-OVERRIDE | rate=0.85 (source=user_preference) → -15% (bandit=fast and confusion=0.30 ignored)
```

Confusion détectée :

```
■■ tts rate composed | profile=1.05 (level_default(université)) × bandit=1.00 (normal) × confusion=0.90 (score=
```

13.7 Conversion finale

```
def rate_float_to_edge_str(rate: float) -> str:
    pct = int(round((rate - 1.0) * 100))
    pct = max(-50, min(50, pct))      # Edge-TTS limit
    return f"{pct:+d}%"
```

rate	Edge-TTS string
0.50	-50%
0.85	-15%
0.92	-8%
1.00	+0%
1.05	+5%
1.15	+15%
1.50	+50%

PARTIE IV — CALCULS DE SCORE

14. Mastery

14.1 Modèle bayésien

Pour chaque (student_id, course_id, idea_id), on stocke 2 compteurs :

- **attempts** : nombre total d'interactions sur cette idée
- **confusions** : sous-ensemble où l'étudiant a montré de la confusion

Prior : $\text{Beta}(1, 1)$ (uniforme sur $[0, 1]$).

Likelihood (modèle Bernoulli) : chaque attempt est un essai indépendant avec succès probabilité θ , où "succès" = pas de confusion.

Posterior : $\text{Beta}(1 + \text{correct}, 1 + \text{confusions})$ avec $\text{correct} = \text{attempts} - \text{confusions}$.

Estimateur (moyenne posterior) — Laplace rule of succession :

$$\theta_{\text{est}} = (\text{correct} + 1) / (\text{attempts} + 2)$$

14.2 Implémentation

```
def _laplace_score(attempts: int, confusions: int) -> float:
    """Posterior mean under Beta(1,1) prior – Laplace rule of succession."""
    correct = max(0, attempts - confusions)
    return (correct + 1) / (attempts + 2)
```

14.3 Worked examples

attempts	confusions	correct	θ_{est} formule	Valeur
0	0	0	$(0+1)/(0+2)$	0.500 (cold start = uniform prior)
1	0	1	$(1+1)/(1+2)$	0.667
1	1	0	$(0+1)/(1+2)$	0.333
5	0	5	$(5+1)/(5+2)$	0.857
5	1	4	$(4+1)/(5+2)$	0.714
5	2	3	$(3+1)/(5+2)$	0.571
5	3	2	$(2+1)/(5+2)$	0.429
10	0	10	$(10+1)/(10+2)$	0.917
10	5	5	$(5+1)/(10+2)$	0.500
50	5	45	$(45+1)/(50+2)$	0.885
50	25	25	$(25+1)/(50+2)$	0.500
100	0	100	$(100+1)/(100+2)$	0.990

14.4 Pourquoi Laplace plutôt que MLE

Maximum Likelihood Estimator naïf : $\text{correct} / \text{attempts}$.

Cas	MLE	Laplace
0/0	undefined / NaN	0.500 (gracieux)
1/1	1.0 (sur-confiant)	0.667
5/5	1.0	0.857
0/1	0.0 (sur-pessimiste)	0.333

Laplace évite les zéros et uns absolus → robuste sur petits N, moins prone aux sur-réactions.

14.5 Update flow

```

async def update_mastery(student_id, course_id, idea_id, is_confusion: bool):
    # Read current
    row = await db.execute(
        select(StudentMastery).where(
            StudentMastery.student_id == sid,
            StudentMastery.course_id == cid,
            StudentMastery.idea_id == idea_id,
        )
    ).first()

    if row is None:
        row = StudentMastery(
            student_id=sid, course_id=cid, idea_id=idea_id,
            attempts=0, confusions=0, score=0.5,
        )
        db.add(row)

    row.attempts += 1
    if is_confusion:
        row.confusions += 1
    row.score = _laplace_score(row.attempts, row.confusions)
    row.last_seen_at = datetime.utcnow()
    await db.commit()

```

14.6 Logs

```

■ mastery UPDATE | idea=k_means_centroid | CLEAN | correct=4/6 (confusions=2) | Beta(1,1)+Laplace : (4+1)/(6+2)
■ mastery UPDATE | idea=k_means_centroid | CONFUSION | correct=4/7 (confusions=3) | Beta(1,1)+Laplace : (4+1)/(7+2)

```

15. FSRS

Source : `pedagogy/review_scheduler.py` + lib `fsrs` (PyPI).

FSRS (Free Spaced Repetition Scheduler) maintient pour chaque `(student, concept_name)` :

Champ	Type	Sens
<code>stability (S)</code>	<code>float</code>	Combien de jours avant que l'étudiant oublie (en théorie : intervalle où retention = 90%)
<code>difficulty (D)</code>	<code>float ∈ [1, 10]</code>	À quel point le concept est difficile pour cet étudiant
<code>state</code>	<code>enum</code>	Learning / Review / Relearning
<code>due</code>	<code>datetime</code>	Date du prochain rappel
<code>step</code>	<code>int</code>	Position dans la sequence d'apprentissage initial
<code>last_review</code>	<code>datetime</code>	

Champ	Type	Sens
review_count	int	
lapse_count	int	Nombre d'oublis (rating=Again)

15.1 Ratings

Rating	Sémantique	Effet
Again = 1	Fausse réponse	S diminue, D augmente, lapse_count++
Hard = 2	Correct mais avec hints	S augmente peu
Good = 3	Correct normalement	S augmente significativement
Easy = 4	Parfait, instant	S augmente beaucoup

15.2 Formules FSRS-4.5 (simplified)

L'algorithme FSRS-4.5 utilise des poids $w[0..16]$ appris sur dataset Anki (<https://github.com/open-spaced-repetition/fsrs4anki>). Les formules clés :

Recall probability at time t after last review :

$$R(t) = (1 + t / (S \times \text{FACTOR}))^{(-1)}$$

où $\text{FACTOR} = 19/81 \approx 0.235$ (calibrated).

Stability update after correct review (rating $G \in \{\text{Hard, Good, Easy}\}$) :

$$S_{\text{new}} = S_{\text{old}} \times (1 + \exp(w[8]) \times (11 - D) \times \text{pow}(S_{\text{old}}, -w[9]) \times (\exp((1 - R) \times w[10]) - 1) \times \text{hard_penalty}(G) \times$$

Stability update after lapse (rating = Again) :

$$S_{\text{new}} = w[11] \times \text{pow}(D, -w[12]) \times (\text{pow}(S_{\text{old}} + 1, w[13]) - 1) \times \exp((1 - R) \times w[14])$$

Difficulty update :

$$D_{\text{new}} = D_{\text{old}} + w[6] \times (G - 3)$$

$$D_{\text{new}} = \text{clamp}(D_{\text{new}}, 1, 10)$$

$$D_{\text{new}} = w[7] \times D_{\text{initial}} + (1 - w[7]) \times D_{\text{new}} \quad \# \text{ mean reversion}$$

Next interval for desired retention = 0.9 :

$$\text{interval_days} = S_{\text{new}} \times (\text{pow}(0.9, -1/\text{FACTOR}) - 1)$$

15.3 Logs

- FSRS UPDATE | concept='k_means' rating=good | stability=4.20 difficulty=0.45 state=review | review_count=3 la
- Review scheduled: student=ce566ac0 concept='k_means' rating=good next_due=2026-05-11T10:00:00

15.4 Listing due reviews

`ReviewScheduler.list_due(student_id, course_id, limit=20) :`

```
SELECT * FROM review_queue
WHERE student_id = $1
  AND due <= NOW()
ORDER BY due ASC
```

LIMIT \$2

Filter par course via KG lookup (concept_name → ConceptInfo.course_id).

16. Bandit Thompson sampling

16.1 Algorithmme

```
def select_arm(arms_state: dict[str, ArmState], rng=None) -> str:
    rng = rng or random.Random()
    samples = {}
    for arm_name, arm in arms_state.items():
        # Beta( $\alpha$ ,  $\beta$ ) sample
        samples[arm_name] = rng.betavariate(arm.alpha, arm.beta)
    return max(samples, key=samples.get)
```

16.2 Justification mathématique

Thompson sampling minimise le regret bayésien (Russo & Van Roy 2014). Garantie : $\text{regret}(T) = O(\sqrt{K T \log T})$ pour K bras et T turns, optimal à un facteur constant près.

Intuition : sampler $\theta_i \sim \text{Beta}(\alpha_i, \beta_i)$ puis prendre l'argmax équivaut à prendre l'arm probablement optimal. Avec N pulls, la variance Beta diminue → samples se concentrent autour de la moyenne → exploitation. Avant ça → exploration.

16.3 Cold start

Tous arms commencent à $\alpha=\beta=1$ → samples uniformes sur [0, 1] → exploration aléatoire.

Avec N=1 pull et reward=1 :

- $\alpha=2, \beta=1 \rightarrow$ moyenne = $2/3 \approx 0.67$
- Variance Beta(2,1) = $2/(9 \cdot 4) \approx 0.056$ (large)

Avec N=10 pulls et reward moyen = 0.7 :

- $\alpha \approx 8, \beta \approx 4 \rightarrow$ moyenne = $8/12 \approx 0.67$
- Variance Beta(8,4) ≈ 0.017 (plus serré)

16.4 Worked example

Bucket mixed|fast|medium|isolated, 18 arms, après 5 turns :

Arm	α	β	mean	Var
socratic:slow	4.0	2.0	0.667	0.032
simpler_words:fast	2.0	4.0	0.333	0.032
decomposition:fast	1.0	1.0	0.500	0.083
analogy:normal	1.5	1.5	0.500	0.063
...				

Au turn 6, samples (RNG seeded) :

- socratic:slow $\rightarrow$ 0.997 $\leftarrow$ chosen
- simpler_words:fast $\rightarrow$ 0.966
- decomposition:fast $\rightarrow$ 0.899
- ...

(Note : socratic:slow avec mean 0.67 a souvent le sample le plus haut grâce à $\alpha >> \beta$.)

17. Bandit reward

Source : `pedagogy/personalization/bandit/reward.py`.

```
reward = W_CONFUSION × confusion_signal  
        + W_MASTERY   × mastery_signal  
        + W_ENGAGEMENT × engagement_signal
```

Avec :

- confusion_signal = 1.0 if NOT confused else 0.0
- mastery_signal = max(0, Δ mastery) clamped to [0, 1] (gain de mastery)
- engagement_signal = 1.0 if engaged else 0.0

Poids actuels (Phase 1) :

- W_CONFUSION = 0.50 (le plus important)
- W_MASTERY = 0.30
- W_ENGAGEMENT = 0.20

Somme = 1.0 $\rightarrow$ reward $\in$ [0, 1].

17.1 Worked examples

confused?	Δ mastery	engaged?	reward
no	0.0	yes	$0.5 \times 1 + 0.3 \times 0 + 0.2 \times 1 =$ 0.70
no	0.10	yes	$0.5 + 0.03 + 0.2 =$ 0.73
no	0.20	no	$0.5 + 0.06 + 0 =$ 0.56
yes	0.0	yes	$0 + 0 + 0.2 =$ 0.20
yes	-0.05	no	$0 + 0 + 0 =$ 0.00
no	0.50	yes	$0.5 + 0.15 + 0.2 =$ 0.85

17.2 Update Beta posterior

```
 $\alpha$ [chosen] += reward  
 $\beta$ [chosen] += (1 - reward)
```

Exemple : reward = 0.7 $\rightarrow$ α += 0.7, β += 0.3.

18. Confusion fusion

Voir §12.3 pour la formule. Worked examples détaillés :

sight	prosody markers	prosody_score	fused	Verdict
0.85	0 (clean)	0.0	$0.7 \times 0.85 + 0.3 \times 0 = 0.595$	clean (< 0.5 ? non, > 0.5 → confused)
0.10	3 (very strong)	1.0	$0.7 \times 0.10 + 0.3 \times 1.0 = 0.37$	clean
0.10	0	0.0	$0.7 \times 0.10 + 0 = 0.07$	clean
0.85	2 (strong)	0.7	$0.7 \times 0.85 + 0.3 \times 0.7 = 0.805$	confused
0.5	1 (mild)	0.30	$0.7 \times 0.5 + 0.3 \times 0.30 = 0.44$	clean
0.5	2	0.70	$0.7 \times 0.5 + 0.3 \times 0.7 = 0.56$	confused
0.65	0	0	(sight only) 0.65	confused
None	2	0.70	(prosody only) 0.70	confused

19. Engagement

Source : `pedagogy/engagement.py`.

19.1 Formule globale

```
score = W_RECENCY × recency_score
      + W_QUESTION × question_score
      + W_CONFUSION × confusion_score
      + W_PASSIVE × passive_score
      + W_INTERRUPT × interrupt_score
```

Avec poids défaut (somme = 1.0) :

- `W_RECENCY` = 0.30
- `W_QUESTION` = 0.25
- `W_CONFUSION` = 0.20
- `W_PASSIVE` = 0.15
- `W_INTERRUPT` = 0.10

19.2 Sub-scores

Recency (`recency_score`) :

```
 $\Delta t$  = seconds_since_last_interaction
Si  $\Delta t$  < 30s → 1.0
Si  $\Delta t$  > 600s → 0.0
Sinon : (600 -  $\Delta t$ ) / 570 # decay linéaire
```

Question (`question_score`) :

```
session_age_min = max(0.5, session_age_s / 60)
q_per_min      = questions_in_session / session_age_min
```

Goldilocks zone : optimal à 0.5 q/min (Fredricks 2004).

- 0.5 q/min → 1.0
- 0.0 q/min → 0.3 (silence ambigu)
- 2.0 q/min → 0.7 (trop = struggling)
- ≥4.0 → 0.5

Confusion (confusion_score) :

```
confusions_in_session
0 → 1.0
1 → 0.7
2 → 0.4    (= ENGAGEMENT_CONFUSION_OPTIMAL : un peu de confusion = challenging)
3 → 0.3
≥4 → 0.2
```

Note : la « confusion optimale » à 2 reflète l'effet de challenge (Csikszentmihalyi flow theory).

Passive (passive_score) :

```
consecutive_passive_slides
0 → 1.0
1 → 0.85
2 → 0.6
3 → 0.45
5 → 0.3
≥10 → 0.1
```

Interrupt latency (interrupt_score) :

```
last_interrupt_latency_ms
< 200ms → 1.0
500ms   → 0.85
1000ms  → 0.5
2000ms  → 0.0    (passé un certain seuil, pas réactif)
```

19.3 Bucket label

```
score < 0.30 → "disengaged"
score < 0.65 → "neutral"
score ≥ 0.65 → "engaged"
```

19.4 Worked example

Étudiant en début de session, vient de poser une question, 0 confusion, slide 2 sans interaction passive, latency 250ms :

- recency = 1.0 (juste interagi)
- question = 0.6 (1 q en 1 min)
- confusion = 1.0 (0)
- passive = 0.85 (1)
- interrupt = 0.95 (250ms)

```
score = 0.30×1.0 + 0.25×0.6 + 0.20×1.0 + 0.15×0.85 + 0.10×0.95
      = 0.30 + 0.15 + 0.20 + 0.1275 + 0.095
      = 0.873 → "engaged"
```

19.5 Logs

■ engagement | score=0.65 (engaged) | recency=1.00 questions=0.00 confusion=0.50 passive=1.00 interrupt=1.00

20. RAG ranking

Voir §6.4 et §8.2 pour le pipeline. Détails math :

20.1 Cosine similarity (vector search)

Avec embeddings BGE-m3 normalisés (norme = 1), $\text{cosine}(u, v) = u \cdot v$. Range : [-1, 1] mais en pratique [0, 1] pour text embeddings.

20.2 BM25 (lexical search)

Robertson 1995 :

$$\text{BM25}(q, d) = \sum \text{IDF}(q_i) \times ((\text{tf}(q_i, d) \times (k_1 + 1)) / (\text{tf}(q_i, d) + k_1 \times (1 - b + b \times |d| / \text{avgdl})))$$

- $\text{IDF}(q_i) = \log((N - \text{df}(q_i) + 0.5) / (\text{df}(q_i) + 0.5))$
- $\text{tf}(q_i, d)$: fréquence du terme q_i dans doc d
- $|d|$: longueur du doc, avgdl : longueur moyenne
- $k_1 = 1.5, b = 0.75$ (langchain defaults)

20.3 RRF fusion (Cormack 2009)

$$\text{score}(d) = \sum_i \text{weight}_i / (k + \text{rank}_i(d))$$

Smart Teacher l'instancie avec 2 sources (BM25 + dense) :

$$\text{score}(d) = \alpha / (60 + \text{rank}_{\text{bm25}}(d)) + (2 - \alpha) / (60 + \text{rank}_{\text{dense}}(d))$$

α adaptatif :

```
def _bm25_alpha(query: str) -> float:
    tokens = tokenize(query)
    avg_idf = mean(idf(t) for t in tokens if t in vocab)
    # Normalize avg_idf to [0, 1] range
    lo, hi = 1.5, 6.0 # empirical bounds
    norm = (avg_idf - lo) / (hi - lo)
    norm = max(0.0, min(1.0, norm))
    return 1.0 + 0.32 * norm # ∈ [1.0, 1.32]
```

20.4 Cross-encoder rerank

Modèle : **BAAI/bge-reranker-v2-m3** (568M params, multilingual).

Input format : **[CLS] query [SEP] doc [SEP]**. Output : score logit, sigmoid → probability ∈ [0, 1].

Plus précis que dual-encoder cosine mais plus lent (~30ms par pair sur GPU, 200ms+ CPU).

Compromis adopté : top 15 candidats RRF → rerank → top 5.

20.5 Logs RAG

- QDRANT search START | collection=smart_teacher_multimodal... k=15 | course=9940c8f4 chapter=1 | filters=2 |
- QDRANT search DONE | hits=15/15 | top_score=0.847 mean=0.692 | took=42ms
- BM25 boost $\alpha=1.32$ (query has rare terms)
- 5 chunks retenus (rerank: 0.62 [cross-encoder], cosine: 0.41 [bge-m3])
- rag[0] rerank=0.731 cosine=n/a ch=1 sec=10 src=? | "La RI ..."
- rag[1] rerank=0.501 cosine=n/a ch=1 sec=15 src=? | "..."

21. Concept aggregation

Source : `pedagogy/student_knowledge.py`.

```
def build_snapshot(student_id, course_id, kg) -> StudentKnowledgeSnapshot:
    concepts = kg.list_concepts(course_id)
    all_idea_ids = [iid for c in concepts for iid in c.idea_ids]
    idea_scores = await MasteryRepo.get_scores_bulk(student_id, course_id, all_idea_ids)

    snap = StudentKnowledgeSnapshot(student_id=student_id, course_id=course_id)
    for c in concepts:
        ids = list(c.idea_ids)
        scored = [idea_scores[i] for i in ids if i in idea_scores]
        if scored:
            mean_score = round(sum(scored) / len(scored), 3)
            snap.mastery_by_concept[c.name] = mean_score

    snap.total_attempts = len(idea_scores)

    # Buckets
    for name, score in snap.mastery_by_concept.items():
        if score >= 0.7:
            snap.strong_concepts.append(name)
        if score < 0.4:
            snap.weak_concepts.append(name)
        if score <= 0.3:
            snap.recently_confused.append(name)

    snap.never_seen_concepts = [
        c.name for c in concepts if c.name not in snap.mastery_by_concept
    ]

    return snap
```

Cold start : si `total_attempts ≤ Config.KNOWLEDGE_COLD_START_ATTEMPTS=5` → snapshot empty
`to_prompt_context()` → "".

21.1 Logs

- snapshot AGG | concept=k_means | mean(0.65, 0.72, 0.58) = 0.650 / 3 ideas
- snapshot AGG | concept=hyperplane | mean(0.42) = 0.420 / 1 ideas
- snapshot | strong=[k_means] weak=[hyperplane] never_seen=[svm]

22. Cursor / sentence-span

Source : `services/presentation.py`.

22.1 Conversion audio_progress → cursor

```
def compute_text_cursor_from_audio_progress(audio_progress: float, narration_text: str) -> int | None:
    if audio_progress is None or not narration_text:
        return None
    progress = float(audio_progress)
```

```

if progress != progress: # NaN
    return None
if progress <= 0.0 or progress > 1.0:
    return None
cursor = int(round(progress * len(narration_text)))
return max(0, min(cursor, len(narration_text)))

```

Approximation linéaire : suppose chars/sec uniforme. Erreur typique < 1 phrase.

22.2 Sentence rewind

Cherche le terminator de phrase précédant **cursor** :

```

def rewind_to_current_sentence_start(narration: str, cursor: int) -> int:
    if cursor <= 0:
        return 0
    if cursor >= len(narration):
        return rewind_to_last_sentence_start(narration)
    head = narration[:cursor]
    terminators = list(re.finditer(r"(?:[.!?...■]\s+|[■][■][■])(?=[\s\S])", head))
    if terminators:
        return terminators[-1].end()
    return 0

```

Multi-script :

- Latin : **[.!?...]**
- Arabic : **■**
- CJK : **[■][■][■]**

22.3 Sentence span (REEXPLAIN)

```

def current_sentence_span(narration: str, cursor: int) -> tuple[int, int, str]:
    n = len(narration)
    pos = max(0, min(cursor, n - 1))
    start = rewind_to_current_sentence_start(narration, pos)
    forward = re.search(r"(?:[.!?...■](?:\s|\n)+|[■][■][■])(?=[\s\S])", narration[start:])
    end = start + forward.end() if forward else n
    return (start, end, narration[start:end].strip())

```

22.4 Logs

- audio_progress→cursor | progress=0.4292 × narration_len=1918 = cursor=823 (42.9% played)
- sentence_span | cursor=823 → start=750 end=910 (len=160) | preview='La RI consiste à trouver des documents.'
- rewind_to_current_sentence_start | cursor=823 → rewound to sentence start=750 (skipped back 73 chars across

PARTIE V — RÉFÉRENCE

23. Configuration

Source : **core/config.py**. Toutes les variables surchargeables via env vars.

23.1 API Keys & LLM providers

Variable	Défaut	Description
OPENAI_API_KEY	None	Clé OpenAI premium
ELEVENLABS_API_KEY	None	Clé ElevenLabs (TTS premium)
GROQ_API_KEY	None	Clé Groq (fallback hosted LLM rapide)
GROQ_BASE_URL	https://api.groq.com/openai/v1	Endpoint OpenAI-compatible
GROQ_MODEL	llama-3.3-70b-versatile	Modèle Groq par défaut
DISABLE_OPENAI	false	Kill-switch OpenAI → Groq/Ollama only
GPT_MODEL	gpt-4o-mini	Modèle OpenAI
GPT_MAX_TOKENS	400	Max tokens par réponse
GPT_TEMPERATURE	0.7	Sampling temp
MAX_HISTORY_TURNS	10	Nb max de turns dans le prompt history

23.2 Database / cache

Variable	Défaut	Description
POSTGRES_HOST	localhost	
POSTGRES_PORT	5432	
POSTGRES_DB	smart_teacher	
POSTGRES_USER	admin	
POSTGRES_PASSWORD	secret	
REDIS_HOST	localhost	
REDIS_PORT	6379	
REDIS_DB	0	
SESSION_TOKEN_TTL	300 (5 min)	One-time WS token TTL
HTTP_HISTORY_TTL	3600 (1h)	REST /ask history TTL
SESSION_TTL	3600 (1h)	SessionContext WS TTL
PRESENTATION_SNAPSHOT_TTL	3600 (1h)	Snapshot par slide
TTS_CACHE_TTL	86400 (24h)	Cache audio TTS phrase
NARRATION_CACHE_TTL	604800 (7j)	Cache narration cross-session
EMBEDDING_CACHE_TTL	86400 (24h)	Cache embeddings

23.3 Audio / STT / TTS

Variable	Défaut	Description
SAMPLE_RATE	16000	Hz audio
CHUNK_SIZE	512	Sample par chunk
SPEECH_THRESHOLD	0.3	VAD threshold

Variable	Défaut	Description
SILENCE_DURATION	1.0	s de silence avant fin de speech
MAX_AUDIO_DURATION	30.0	s max par utterance
STT_BACKEND	faster-whisper	faster-whisper/whisperlivekit
WHISPER_MODEL_SIZE	base	tiny/base/small/medium/large-v3
WHISPER_DEVICE	cpu	cpu/cuda
WHISPER_COMPUTE	int8	int8/float16/float32
WHISPER_THREADS	4	Threads CPU
STT_MIN_AUDIO_SEC	0.1	Filtrer audio trop courts
STT_BEAM_SIZE	3	Beam search width
TTS_PROVIDER	edge	edge/elevenlabs
TTS_VOICE	default	Voix par défaut
TTS_OUTPUT_FORMAT	mp3_22050_32	

23.4 RAG

Variable	Défaut	Description
RAG_ENABLED	true	Master switch RAG
RAG_NUM_RESULTS	5	Top-K final (after rerank)
RAG_EMBEDDING_MODEL	BAAI/bge-m3	HF model id
RAG_DB_DIR	data/multimodal_db	Cache dir
RAG_USE_RERANKER	true	Activer cross-encoder
RAG_RERANKER_MODEL	BAAI/bge-reranker-v2-m3	
RAG_RERANKER_TOP_N	15	Candidats au reranker
RAG_USE_IDEA_CHUNKING	false	LLM idea-segment (off : Ollama hallucine)
RAG_IDEAS_PER_CHUNK_MAX	6	Si idea_chunking on
RAG_IDEA_MIN_LENGTH	30	Filtre micro-ideas
RAG_LLM_PARALLELISM	6	Threads LLM ingestion
RAG_FUSION_MIN_CHARS	300	Fusion sections courtes
RAG_FUSION_MAX_CHARS	2400	
RAG_USE_GRAPH_EXPANSION	true	KG-aug retrieval
KG_DISABLE_LLM_ENRICH	true	Skip LLM concept enrich (anti-halluc.)

23.5 Pédagogie & resume

Variable	Défaut	Description
RESUME_QUICK_PAUSE_S	10	Seuil pause QUICK
RESUME_NORMAL_PAUSE_S	60	Seuil NORMAL
RESUME_LONG_PAUSE_S	180	Seuil GAP/LONG

Variable	Défaut	Description
RESUME_SLIDE_DONE_GRACE_S	30	Seuil SLIDE_COMPLETED vs REVIEW
KNOWLEDGE_STRONG_THRESHOLD	0.7	Mastery → strong
KNOWLEDGE_WEAK_THRESHOLD	0.4	Mastery → weak
KNOWLEDGE_CONFUSED_THRESHOLD	0.3	Mastery → recently_confused
KNOWLEDGE_MAX_LISTED_CONCEPTS	8	Cap pour prompt
KNOWLEDGE_COLD_START_ATTEMPTS	5	Cold start cutoff
CHAT_HISTORY_MAX_TURNS	30	Bound prompt chat history
CHAT_HISTORY_TTL_S	2592000 (30j)	Persistance chat
GROUNDING_OVERLAP_THRESHOLD	0.08	Off-topic guardrail
SPEECH_RATE_SLOW_DOWN_FACTOR	0.85	Voice nav "ralentir"
SPEECH_RATE_FLOOR	0.5	Borne basse

23.6 Engagement (poids somme=1.0)

Variable	Défaut	Description
ENGAGEMENT_W_RECENCY	0.30	
ENGAGEMENT_W_QUESTION	0.25	
ENGAGEMENT_W_CONFUSION	0.20	
ENGAGEMENT_W_PASSIVE	0.15	
ENGAGEMENT_W_INTERRUPT	0.10	
ENGAGEMENT_RECENT_FULL_S	30	Full credit window
ENGAGEMENT_FULLY_STALE_S	600	Decay endpoint
ENGAGEMENT_CONFUSION_OPTIMAL	2	Goldilocks (challenge)
ENGAGEMENT_INTERRUPT_FAST_MS	200	
ENGAGEMENT_INTERRUPT_SLOW_MS	2000	
ENGAGEMENT_DISENGAGED_BELOW	0.30	
ENGAGEMENT_ENGAGED_ABOVE	0.65	

23.7 Anti-écho audio

Variable	Défaut	Description
TTS_SUPPRESSION_WINDOW_S	1.5	Fenêtre suppression mic après TTS
TTS_ECHO_TAIL_S	0.5	Tail echo decay
MIN_INTERRUPT_DURATION_S	0.4	Min voix avant accept interrupt
MIN_INTERRUPT_BYTES	3500	Min bytes audio
MIN_TURN_ENERGY_RMS	0.015	Min RMS énergie
ENABLE_ECHO_XCORR	true	Cross-corrélation anti-écho
ECHO_XCORR_THRESHOLD	0.55	Seuil xcorr
ECHO_XCORR_BUFFER_S	5.0	Buffer ring PCM

23.8 JWT auth

Variable	Défaut	Description
JWT_SECRET_KEY	(placeholder, À CHANGER en prod)	Clé HS256
JWT_ALGORITHM	HS256	
JWT_EXPIRATION_HOURS	24	TTL token

23.9 Confusion model

Variable	Défaut	Description
CONFUSION_MODEL_PATH	dataset/sight-main/.../confusion_model_final.pth	SIGHT XLM-R weights

23.10 Stockage externe

Variable	Défaut	Description
MINIO_ENDPOINT	(vide)	Vide = local fs
MINIO_ACCESS_KEY	minioadmin	
MINIO_SECRET_KEY	minioadmin	
MINIO_BUCKET	smart-teacher	
MINIO_SECURE	false	TLS MinIO
LOCAL_MEDIA_DIR	./media	
CLICKHOUSE_HOST	localhost	
CLICKHOUSE_PORT	8123	
CLICKHOUSE_DB	smart_teacher	
CLICKHOUSE_USER	default	

Variable	Défaut	Description
CLICKHOUSE_PASSWORD	(vide)	
QDRANT_HOST	localhost	
QDRANT_PORT	6333	
QDRANT_COLLECTION	smart_teacher_multi modal	Préfixe collection

23.11 Server

Variable	Défaut	Description
SERVER_HOST	0.0.0.0	
SERVER_PORT	8000	
MAX_RESPONSE_TIME	5.0	s SLA réponse end-to-end
TARGET_RTF	0.50	Real-Time Factor STT

23.12 OCR / Vision LLM

Variable	Défaut	Description
OCR_MIN_IMAGE_BYTES	2000	
OCR_MIN_IMAGE_DIM_PX	50	
USE_VISION_LLM	false	
VISION_LLM_MODEL	gpt-4o-mini	
LIBREOFFICE_BIN	libreoffice	
VISION_DESCRIBE_ENABLED	true	
OLLAMA_VISION_MODEL	llava	
OLLAMA_URL	http://localhost:11434	
OLLAMA_NUM_THREADS	0 (auto)	
DISABLE_VISION_TITLES	false	
INGESTION_VERBOSE_LOGS	false	

24. API REST

24.1 Auth

POST /auth/register

POST /auth/register HTTP/1.1
Content-Type: application/json

```
{
  "email": "alice@example.com",
  "password": "Strong123",
  "first_name": "Alice",
}
```

```
"last_name": "Doe",
"preferred_language": "fr",
"student_level": "lycée"
}
```

Response 201 :

```
{
  "access_token": "eyJ...",
  "token_type": "bearer",
  "student_id": "ce566ac0-...",
  "email": "alice@example.com",
  "account_level": "student",
  "first_name": "Alice"
}
```

POST /auth/login

POST /auth/login HTTP/1.1
Content-Type: application/json

```
{"email": "alice@example.com", "password": "Strong123"}
```

Response : même que register.

POST /auth/logout

Clear cookie. Response : `{"status": "ok"}`.

GET /auth/me

GET /auth/me HTTP/1.1
Cookie: smart_teacher_token=eyJ...

Response :

```
{
  "student_id": "ce566ac0-...",
  "email": "alice@example.com",
  "account_level": "student",
  "first_name": "Alice",
  "last_name": "Doe",
  "preferred_language": "fr",
  "student_level": "lycée",
  "is_active": true
}
```

24.2 Course

POST /course/build

Multipart upload. Form fields :

- **files**: list of UploadFile (PDF/DOCX/PPTX)
- **language**: str (default "fr")
- **level**: str (default "lycée")
- **domain**: str (default "general")

Response :

```
{
```

```

"results": [{
  "file": "Chapitre 1.pdf",
  "course_id": "9940c8f4-...",
  "title": "Chapitre 1 Introduction A La Ri",
  "chapters": 1,
  "sections": 19,
  "domain": "informatique",
  "course": "recherche_information",
  "chapter": "chapitre_1_introduction_a_la_ri",
  "storage_path": "courses/.../Chapitre 1.pdf",
  "status": "ok",
  "db_error": null
}],
"rag_stats": {
  "total_docs": 64,
  "embedding_dim": 1024,
  ...
}
}

```

GET /course/list

Response :

```

{"courses": [
  {"id": "9940c8f4-...", "title": "Chapitre 1 ...", "subject": "recherche_information", "language": "fr", "level": "..."}
]}

```

GET /course/{id}/structure

Response :

```

{
  "course": {"id": "...", "title": "...", "language": "fr", "level": "..."},
  "chapters": [
    {"id": "...", "title": "...", "order": 1, "sections": [
      {"id": "...", "title": "...", "order": 1, "image_url": "...", "char_count": 487}
    ]}
  ]
}

```

GET /course/{id}/concept-graph

Response (Cytoscape format) :

```

{
  "nodes": [{"data": {"id": "k_means", "label": "K-means", "score": 12.0, "bloom": "apply"}}, ...],
  "edges": [{"data": {"source": "k_means", "target": "centroid", "type": "prereq", "weight": 3}}, ...]
}

```

24.3 Session

GET /session/{session_id}/profile

Response :

```

{"status": "ok", "profile": {...}}

```

POST /session/{session_id}/profile

```

{"level": "université", "preferences": {"speech_rate": 1.10}}

```

GET /session/{session_id}/tts_params

Query : ?confusion_score=0.3 (optional)

Response :

```
{
  "status": "ok",
  "tts_params": {
    "rate": 1.05,
    "pitch": 1.0,
    "voice": "default",
    "rate_source": "level_default(université)",
    "manual_override": false
  }
}
```

POST /session/{session_id}/speech_rate

```
{
  "rate": 0.85,
  "manual_override": true
}
```

Response :

```
{
  "status": "ok",
  "rate": 0.85,
  "edge_tts_rate": "-15%",
  "manual_override": true,
  "rate_source": "user_preference"
}
```

24.4 Student

GET /student/me/state

Response :

```
{
  "student_id": "ce566ac0-...",
  "courses_seen": 1,
  "total_attempts": 42,
  "mastery_avg": 0.62,
  "confusion_count": 8,
  "engagement": {"score": 0.72, "label": "engaged"},
  "due_reviews": 3,
  "knowledge_snapshot": {
    "strong": ["k_means"],
    "weak": ["em_algorithm"],
    "never_seen": ["dbscan"],
    "recently_confused": []
  }
}
```

GET /student/me/learning-style

Response :

```
{
  "posterior": {"visual": 0.45, "verbal": 0.30, "kinesthetic": 0.25},
  "dominant": "visual",
  "confidence": 0.45
}
```

POST /student/me/learning-style

VARK self-report :

```
{
  "answers": [{"question_id": "Q1", "choice": "visual"}, ...]
}
```

GET /student/me/practice-due

Response :

```
{
  "due_count": 3,
  "concepts": [
    {"name": "k_means", "due": "2026-05-04T18:00:00Z", "stability": 4.2, "difficulty": 0.45},
    ...
  ]
}
```

24.5 Misc

POST /ask (one-shot Q&A REST)

```
{"question": "Qu'est-ce que la RI ?", "course_id": "9940c8f4-...", "session_id": "..."}"
```

Response :

```
{
  "answer": "La RI consiste à...",
  "confidence": 0.78,
  "supporting_chunks": [{"id": "...", "text": "...", "source": "..."}],
  "performance": {"total_time": 2.34, "stt_time": 0, "llm_time": 1.32, "tts_time": 0}
}
```

GET /health

```
{"status": "ok", "uptime_s": 3600, "services": {"postgres": "up", "redis": "up", "qdrant": "up"}}
```

25. WebSocket protocol

URL : `ws://localhost:8000/ws/{session_id}`

Auth : JWT cookie `smart_teacher_token` (extrait via cookie ou Authorization header).

25.1 Client → Server

start_session

```
{"type": "start_session", "course_id": "9940c8f4-...", "language": "fr", "level": "lycée"}
```

present_section

```
{
  "type": "present_section",
  "presentation_request_id": "uuid",
  "course_id": "9940c8f4-...",
  "course_title": "Chapitre 1 Introduction A La Ri",
  "course_domain": "informatique",
  "chapter": "Chapitre 1: Introduction A La Ri",
  "chapter_index": 1,
  "section_title": "II. Définitions et terminologie",
  "section_index": 3,
  "slide_index": 3,
  "slide_title": "II. Définitions et terminologie",
  "slide_content": "Texte OCR de la slide...",
  "keywords": ["RI", "définition", "terminologie"],
  "language": "fr",
  "progress_pct": 16
}
```

```
}
```

interrupt

```
{  
  "type": "interrupt",  
  "reason": "pause", // pause | navigation_next | navigation_prev | repeat | question  
  "audio_progress": 0.4291589702760085,  
  "turn_id": 2  
}
```

text_question

```
{"type": "text_question", "text": "qu'est-ce que la RI ?", "turn_id": 3}
```

audio (binary frames)

Raw PCM int16 little-endian, 16000Hz mono, ~512 samples par frame.

next_section / prev_section / repeat

```
{"type": "next_section"}
```

ping

```
{"type": "ping"}
```

25.2 Server → Client

state

```
{  
  "type": "state",  
  "state": "PRESENTING", // ou LISTENING / PROCESSING / ...  
  "details": {  
    "course_title": "...",  
    "chapter_title": "...",  
    "section_title": "...",  
    "char_position": 0  
  }  
}
```

narration_chunk

```
{  
  "type": "narration_chunk",  
  "seq": 0,  
  "text": "Bonjour et bienvenue...",  
  "audio_b64": "...",  
  "mime": "audio/mpeg",  
  "cursor": 129,  
  "total_chars": 1918  
}
```

transcription

```
{"type": "transcription", "text": "qu'est-ce que la RI", "language": "fr", "confidence": 0.95}
```

intent

```
{"type": "intent", "intent": "question", "confidence": 0.95}
```

confusion_detected

```
{"type": "confusion_detected", "score": 0.72, "reason": "sight_model"}
```

system_notice

```
{"type": "system_notice", "text": "■ Point d'arrêt mémorisé – chapitre 1, section 3, position 471/670."}
```

answer_text

```
{"type": "answer_text", "text": "La RI consiste à...", "subject": "informatique", "turn_id": 3}
```

audio_chunk

Binary MP3 (Edge-TTS).

error

```
{"type": "error", "message": "..."} 
```

pong

```
{"type": "pong"}
```

26. Logs (référence complète)

26.1 Légende emojis

Emoji	Sens
■	Trace de calcul interne (input/output, formule, score)
■	Filtre / rejet (paren duplicate, greeting stripped, ...)
■	Décision de cache / curseur
■	Position ou span calculés
■	Stratégie de resume choisie
■	Pause / silence / no-speech
■	Tirage stochastique (bandit)
■	État pédagogique (snapshot)
■	Message réseau (WebSocket)
■ / ■	Transition d'état / reset
■	Authentification
■	Métrique d'observabilité (KPI)
■	Circuit breaker
■	STT (Whisper)
■	TTS (Edge / ElevenLabs)
■■	Cache hit
■	Pause
■■	Resume
■■	Timing / pause bucket
■	Waiting ticker
■	Diagnostic interrupt

Emoji	Sens
■ ■ / ■ ■	Cursor override / rewind
■	Interruption
■ ■	TTS rate adapter
■	Confusion detection
■	Engagement
■	Storage (MinIO/local), cache hit
■	WebSocket connect/disconnect
■	Slide lookup

26.2 Légende emojis services

Emoji	Service
■	Redis (cache, sessions, snapshots, TTS phrases) ou Groq (LLM)
■	PostgreSQL
■	Qdrant (vecteurs RAG)
■	Ollama (LLM local Mistral)
■	Elasticsearch (full-text transcripts)
■	ClickHouse (analytics)
■	RAM / mémoire in-process

26.3 Logs notables

```
# Boot
■ PG ping OK | took=12ms | url=postgresql+asyncpg://admin:***@localhost:5432/smart_teacher
■ REDIS connect | localhost:6379 | decode_responses=True
■ QDRANT collection_check | name=smart_teacher_multimodal_... exists=True | dim=1024 distance=COSINE
■ WHISPER LOAD START | model_size=base device=cpu compute_type=int8
■ WHISPER LOAD DONE | model=base | took=4.21s
LLMRouter : ■ Groq activé | model=llama-3.3-70b-versatile base=https://api.groq.com/openai/v1
■ Brain : Groq activé (OpenAI désactivé) | model=llama-3.3-70b-versatile

# Auth
■ LOGIN OK | sub=ce566ac0 email=... role=student
■ GATE PASS | path=/static/index.html | sub=ce566ac0 role=student
■ JWT DECODE OK | sub=ce566ac0 ...

# WebSocket connection
■ WebSocket connecté : b55337f0
■ Session créée : b55337f0 | lang=fr
■ Session démarrée | lang=fr level=université state=LISTENING

# Presentation cache decision
■ cache_decide INPUT | requested=...:0:0 | mem_text=0ch | snap_text=0ch | pause_text=0ch
■ cache_decide MISS | requested=...:0:0 | will regenerate via LLM
■ Narration cache MISS (slide=...:0:0) → Teaching Graph va générer

# LLM call
■ Tentative OpenAI... (note: en réalité Groq quand DISABLE_OPENAI=true)
■ OpenAI OK | 7.43s | lang=fr | 670 chars

# TTS streaming
■ TTS GENERATE START | scope=presentation | chars=96 lang=fr rate=+0%
■ TTS GENERATE DONE | took=1.36s | speech_speed=70ch/s
```

```

■ REDIS SETEX tts_phrase | size=49955B ttl=86400s | audio=37152B chars=96

# Pause
■ interrupt msg | audio_progress=0.7035 reason='pause' current_cursor=670
■ audio_progress→cursor | progress=0.7035 × narration_len=670 = cursor=471 (70.4% played)
■ Interrupt cursor override 670 → 471
■ PAUSE START | cursor=471/670 (70.3%) | timestamp=...
■ WAITING ticker STARTED | reason=pause slide=...:0:0
■ interrupt latency = 87ms
■ Interruption

# Wait ticks
■ WAITING tick | elapsed=15s (0m15s) | bucket=NORMAL
■ WAITING tick | elapsed=60s (1m00s) | bucket=GAP
■ WAITING tick | elapsed=180s (3m00s) | bucket=LONG

# Resume
■ RESUME TRIGGERED | trigger='repandre' | wait_total=42.3s
■ RESUME_SESSION | cursor=471/670 (70.3%) | pause_duration=42.3s
■ detect_resume_intent → normal_resume (bucket=normal)
■ compose_resume_action FINAL | strategy=rewind_sentence
■ rewind_to_current_sentence_start | cursor=471 → start=318

# RAG
■ QDRANT search START | k=15 | course=9940c8f4 chapter=1 | query='qu est ce que la RI ?'
■ QDRANT search DONE | hits=15/15 | top_score=0.847 mean=0.692 | took=42ms
■ BM25 boost  $\alpha=1.32$  (query has rare terms)
■ 5 chunks retenus (rerank: 0.62, cosine: 0.41)
■ rag[0] rerank=0.731 cosine=n/a ch=1 sec=10 src=? | "..."

# Q&A graph
■ intent USER text : '...'
■ intent LLM raw : '{"intent":"question",...}'
■ confusion fusion | sight=0.85 (fires) | prosody=0.00 → fused=0.59 → CONFUSED
■ engagement | score=0.65 (engaged) | recency=1.00 questions=0.00 confusion=0.50 passive=1.00 interrupt=1.00
■ bandit THOMPSON | bucket=mixed|fast|medium|isolated | 18 arms | winner=socratic:slow draw=0.997
■ bandit START | strategy=socratic speech_rate=slow | reasoning="..."
■ personalization | style=visual pace=normal depth=balanced tone=challenging
■ responder PROMPT (intent=question, history=0 turns, chunks=5) === ... === END PROMPT
■ responder LLM RAW (656 chars): ...
■ responder PARSED answer (656 chars), citations=0 : ...
■ guardrail | overlap=0.89 (threshold=0.08) | answer_words=19 source_words=51 | common=17
■ reviewer LLM raw : '{"grounded": true, "feedback": ""}'
■ reviewer VERDICT | grounded=True | feedback=''

# TTS rate
■ tts rate composed | profile=1.00 (level_default(lycée)) × bandit=1.15 (fast) × confusion=1.00 = 1.15 → +15%
■ tts rate USER-OVERRIDE | rate=0.85 (source=user-preference) → -15% (bandit=fast and confusion=0.30 ignored)

# Mastery / FSRS
■ mastery UPDATE | idea=k_means | CLEAN | correct=4/6 (confusions=2) | Beta(1,1)+Laplace : (4+1)/(6+2) = 0.625
■ FSRS UPDATE | concept='k_means' rating=good | stability=4.20 difficulty=0.45 state=review | next_due=2026-05-

# Snapshot
■ snapshot AGG | concept=k_means | mean(0.65, 0.72, 0.58) = 0.650 / 3 ideas
■ snapshot | strong=[k_means] weak=[hyperplane] never_seen=[svm]

# Storage
■ STORAGE upload START | backend=local object=audio/sess_abc/1746.mp3 size=145200B
■ STORAGE upload DONE | took=87ms throughput=1630KB/s | url=...
■ ES INDEX | index=transcripts role=student lang=fr | text_len=87 | took=8ms
■ CLICKHOUSE INSERT | table=learning_events lang=fr subj=informatique | took=5ms

```

PARTIE VI — OPÉRATIONS

27. Deployment

27.1 Docker compose (extrait)

```
services:
  smart_teacher_app:
    build: .
    ports: ["8000:8000"]
    depends_on: [postgres, redis, qdrant]
    environment:
      - POSTGRES_HOST=postgres
      - REDIS_HOST=redis
      - QDRANT_HOST=qdrant
      - GROQ_API_KEY=${GROQ_API_KEY}
      - DISABLE_OPENAI=true
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
    volumes:
      - ./media:/app/media
      - ./logs:/app/logs

  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: smart_teacher
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports: ["5432:5432"]
    volumes: ["pg_data:/var/lib/postgresql/data"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333", "6334:6334"]
    volumes: ["qdrant_data:/qdrant/storage"]

  ollama:
    image: ollama/ollama:latest
    ports: ["11434:11434"]
    volumes: ["ollama_data:/root/.ollama"]

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.x
    environment: ["discovery.type=single-node", "xpack.security.enabled=false"]
    ports: ["9200:9200"]

  clickhouse:
    image: clickhouse/clickhouse-server:latest
    ports: ["8123:8123"]

  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    ports: ["9000:9000", "9001:9001"]

volumes:
  pg_data: ~
  qdrant_data: ~
  ollama_data: ~
```

27.2 .env minimal

```
# Critical
POSTGRES_PASSWORD=secret
JWT_SECRET_KEY=change_me_in_production_64_random_bytes
```

```
# LLM (recommended)
GROQ_API_KEY=gsk_...
DISABLE_OPENAI=true

# Optional
ELEVENLABS_API_KEY=...
```

27.3 Boot sequence

1. Postgres + Redis + Qdrant démarrent 2. App container : `python main.py` →

- Validate Config
- Init logging
- Connect Postgres (lazy migrations)
- Connect Redis
- Connect Qdrant + load BGE-m3 embeddings + reranker
- Build IdeaGraph from cached docs
- Load SIGHT model (XLM-R, ~300MB)
- Load VAD Silero
- Init MediaStorage (MinIO ou local)
- Init Elasticsearch (ou fallback)
- Init ClickHouse (ou fallback)
- Compile LangGraphs (qa + teaching)
- Start uvicorn

Total : 30-90s suivant cold/warm caches.

28. Performance & SLA

28.1 KPIs

Métrique	Target	Mesuré
Q&A end-to-end	< 5s	2-7s avec Groq
Q&A end-to-end (Ollama only)	—	60-600s ■
Interrupt latency	< 500ms	80-1300ms (varie selon TTS in-flight)
TTS first chunk	< 2s	1-3s
RTF (real-time factor) STT	< 0.5	0.2-0.4 (Whisper base CPU int8)
RAG retrieval	< 3s	0.5-2.5s

28.2 Throughput typique

Op	Latence
Redis SETEX (~2KB)	1-5ms

Op	Latence
Redis GET	0.5-3ms
Postgres simple query	5-30ms
Qdrant search (k=15)	30-100ms
BGE-m3 embed (1 query)	50-200ms (CPU)
Cross-encoder rerank (15 pairs)	200-800ms (CPU)
Edge-TTS phrase (~150 chars)	1-3s
Groq llama-3.3-70b (~600 tokens out)	0.8-2.5s
OpenAI gpt-4o-mini (~400 tokens out)	0.5-2s
Ollama Mistral CPU (~600 tokens)	60-600s ■■

28.3 Coûts d'API estimés

Provider	Cost/M tokens (input)	Cost/M tokens (output)	Latence median
Groq llama-3.3-70b	gratuit (tier dev)	gratuit	1.2s
OpenAI gpt-4o-mini	\$0.15	\$0.60	1.0s
ElevenLabs (multilingual)	\$0.30 / 1k chars	—	0.8s
Edge-TTS	gratuit	—	1.5s
OpenAI vision (gpt-4o-mini)	\$0.15 / image	—	2-5s

29. Troubleshooting / FAQ

Q1 — Pourquoi le narrateur finit-il par des questions rhétoriques ?

A : tendance naturelle des modèles Llama à être « pédagogue engagé ». Le prompt a été renforcé (cf. §7.5) : **INTERDIT : NE TERMINE JAMAIS par une question rhétorique ouverte. Termine par UNE PHRASE AFFIRMATIVE.**

Q2 — Le RAG retourne 0 chunks alors que le cours est ingéré

Causes possibles : 1. Mismatch course_id : Postgres a un UUID différent de Qdrant. Solution : `python : ``python from qdrant_client import QdrantClient c = QdrantClient(host='localhost', port=6333) for col in c.get_collections().collections: points, _ = c.scroll(col.name, limit=1) print(col.name, points[0].payload.get('metadata', {}).get('course')) ``` Compare avec `SELECT id FROM courses;`. Si différent → re-ingest ou migrer metadata. 2. Filter chapter_idx trop strict : essayer `strict_chapter=False`. 3. BGE-m3 pas chargé : vérifier ■ Local embeddings ready (BAAI/bge-m3) au boot.

Q3 — Pourquoi mes chunks Qdrant ont des définitions hallucinées ?

A : `RAG_USE_IDEA_CHUNKING=true` + Ollama → Mistral hallucine ("RI" → "Research Interface"). Solution : `RAG_USE_IDEA_CHUNKING=false` (default), re-ingest. Ou utiliser un LLM hosté fiable (Groq/OpenAI).

Q4 — Le concept `canonical_name` est faux

A : `KG_DISABLE_LLM_ENRICH=true` (default) — utilise le `section_title` brut du PDF. Pour activer : `false` + Groq/OpenAI configuré.

Q5 — La pause longue ne déclenche pas REEXPLAIN

A : vérifier `RESUME_LONG_PAUSE_S=180` (3 min). Si pause < 3 min → bucket=GAP → REEXPLAIN_AND_CONTINUE déjà déclenché en fait. Vérifier les logs ■ `compose_resume_action FINAL | strategy=...`

Q6 — Comment changer manuellement le débit de parole ?

A :

```
curl -X POST http://localhost:8000/session/{session_id}/speech_rate \
  -H "Content-Type: application/json" \
  -d '{"rate": 0.85, "manual_override": true}'
```

Pour rétablir l'auto : `{"manual_override": false}`.

Q7 — Les WAITING ticks apparaissent en double

A : bug connu — chaque pause démarre un ticker, et l'ancien n'est pas toujours annulé proprement si `pause_session` est appelé deux fois rapidement. Effet cosmétique seulement (les deux tickers convergent).

Q8 — Comment tout vider et repartir propre ?

A :

```
# Stop app
docker stop smart_teacher_app
```

```
# Wipe DB + Qdrant + Redis + disk caches
python scripts/reset_for_reingest.py --apply
```

```
# OR step-by-step:
```

```
python -c "
import asyncio
from sqlalchemy import text
from database.init_db import engine
```

```
async def main():
    async with engine.begin() as conn:
        await conn.execute(text('TRUNCATE students, courses, chapters, sections, ... RESTART IDENTITY CASCADE'))
    asyncio.run(main())
"
```

```
# Restart
docker start smart_teacher_app
```

Q9 — Comment changer le LLM utilisé ?

A : edit `.env` :

```
GROQ_API_KEY=gsk... # primary
```

```
GROQ_MODEL=llama-3.3-70b-versatile # default
DISABLE_OPENAI=true                # skip OpenAI
```

Ou pour OpenAI premium :

```
OPENAI_API_KEY=sk-...
GPT_MODEL=gpt-4o-mini
DISABLE_OPENAI=false
```

Q10 — Performance Ollama insuffisante

Ollama sur CPU : 60-600s par appel LLM = inutilisable pour Q&A interactif. Solutions : 1. Activer Groq (free tier) → 1-2s 2. Activer OpenAI (\$0.15-\$0.60/M tokens) 3. GPU local + Ollama → ~20-50 tok/s sur RTX 3090

Q11 — Comment ajouter une nouvelle stratégie au bandit ?

A : modifier `pedagogy/personalization/bandit/controller.py` :

```
STRATEGIES = ["socratic", "simpler_words", "decomposition", "analogy", "recap", "example", "NEW_STRATEGY"]
```

Et ajouter un mapping `NEW_STRATEGY` → `reasoning_text` dans le code de génération du prompt. Les arms se créent automatiquement avec prior `Beta(1, 1)` au premier sample.

Q12 — Auth gate redirige même avec un token valide

Causes : 1. Cookie `smart_teacher_token` non envoyé : vérifier `Cookie:` header dans devtools 2. JWT expiré : décoder à <https://jwt.io> et vérifier `exp` 3. `JWT_SECRET_KEY` différent entre login et gate (par exemple si redémarrage avec nouvelle clé)

Fin du document.

Version 2026-05-04 — Smart Teacher Architecture & Scores Reference (Detailed Edition)